

UNIVERSIDAD
SERGIO ARBOLEDA

Un simulador a la medida para redes descentralizadas: cobertura y resiliencia de una malla B.A.T.M.A.N.

Juan Camilo Gallardo Sánchez
Jefferson Gutiérrez Tovar

Universidad Sergio Arboleda
Escuela de Ciencias Exactas e Ingeniería
Ingeniería de Ciencias de la Computación e Inteligencia Artificial
Bogotá D.C.

2026

Un simulador a la medida para redes descentralizadas: cobertura y resiliencia de una malla B.A.T.M.A.N.

Juan Camilo Gallardo Sánchez
Jefferson Gutiérrez Tovar

Trabajo de grado

Tutor
Joaquín Fernando Sánchez Cifuentes

Universidad Sergio Arboleda
Escuela de Ciencias Exactas e Ingeniería
Ingeniería de Ciencias de la Computación e Inteligencia Artificial
Bogotá D.C.
2026

Nota de aceptación:

Firma del presidente del jurado

Firma del jurado

Firma del jurado

UNIVERSIDAD
SERGIO ARBOLEDA

Dedicatoria

[Dedicatoria por escribir]

UNIVERSIDAD
SERGIO ARBOLEDA

Agradecimientos

Al profesor Joaquín Fernando Sánchez Cifuentes, tutor de este trabajo, por orientar el proyecto hacia la pregunta que le da sentido: la necesidad de un simulador hecho a la medida para redes descentralizadas.

Al programa de Ingeniería de Ciencias de la Computación e Inteligencia Artificial de la Universidad Sergio Arboleda, en cuyo marco se desarrolló este trabajo.

UNIVERSIDAD
SERGIO ARBOLEDA

Resumen

Las redes descentralizadas no tienen un punto desde el cual se pueda observar o controlar el sistema: cada nodo decide con lo que oye. Evaluarlas exige una herramienta que respete esa condición. Los simuladores de propósito general evalúan un modelo del protocolo reimplementado dentro de la herramienta; los emuladores ejecutan código real con un costo por nodo alto y sin un modelo listo de atenuación entre pisos, y las herramientas de redes entre pares no modelan el medio radio. Este trabajo plantea la necesidad de un simulador hecho a la medida y lo construye: una herramienta en Python que ejecuta en cada nodo simulado las mismas clases de enrutamiento B.A.T.M.A.N. y de detección de fallos del nodo de red, y reemplaza únicamente los sockets por un modelo del medio con distancia y pisos. Sobre esa base se implementó un plano de datos que avanza salto a salto por la tabla de cada nodo, una movilidad de Gateway que decide solo con lo oído, métricas de descentralización, caídas programables, ejecución por lotes con semillas y una interfaz web local.

Como caso de aplicación se evaluó una Red de Expansión de Cobertura con 7 Gateway y 6 Nodos de usuario en un edificio de tres pisos, con 10 semillas por experimento. La caída de un Gateway puente mantuvo la malla partida 90 s sin reducir la cobertura media (1.000): la cobertura, por sí sola, no revela el daño. Dos fallos dirigidos al nodo más crítico redujeron la entrega de datos a 0.284 frente a 0.502 de dos fallos al azar (diferencia pareada de 0.218, IC 95 % [0.149, 0.287]), con cobertura estadísticamente indistinguible. Con Gateway móviles, la partición por la caída del puente bajó de 90 s a 6.2 s, a costa de que la cobertura media descendiera a 0.922. La inundación entregó más que B.A.T.M.A.N. en campo abierto, con cerca del doble de transmisiones, pero mucho menos en el edificio (0.430 contra 0.786), donde sus difusiones carecen de reintentos sobre enlaces débiles. Las tablas de resultados se reprodujeron byte a byte en dos máquinas. El simulador no se validó contra hardware ni contra el módulo batman-adv, y su modelo de radio es simple: los resultados describen esta implementación bajo este medio. Esos frentes quedan como trabajo futuro.

Palabras clave: redes descentralizadas, simulación, B.A.T.M.A.N., redes malladas, cobertura, tolerancia a fallos, movilidad descentralizada, reproducibilidad.

UNIVERSIDAD
SERGIO ARBOLEDA

Abstract

Decentralized networks have no vantage point from which the system can be observed or controlled: each node decides based on what it hears. Evaluating them requires a tool that respects this condition. General-purpose simulators evaluate a model of the protocol reimplemented inside the tool; emulators run real code at a high per-node cost and without a ready-made inter-floor attenuation model; and peer-to-peer tools do not model the radio medium. This work argues for a tailor-made simulator and builds one: a Python tool that runs, in every simulated node, the same B.A.T.M.A.N. routing and failure-detection classes as the network node, replacing only the sockets with a medium model based on distance and floors. On that base it adds a data plane that advances hop by hop through each node's own table, Gateway mobility that decides only from what is heard, decentralization metrics, programmable failures, seeded batch execution and a local web interface.

The application case is a Coverage Expansion Network with 7 Gateways and 6 user nodes in a three-storey building, using 10 seeds per experiment. Losing a bridge Gateway kept the mesh partitioned for 90 s without lowering mean coverage (1.000): coverage alone does not reveal the damage. Two failures targeting the most critical node reduced data delivery to 0.284 against 0.502 for two random failures (paired difference 0.218, 95 % CI [0.149, 0.287]), with statistically indistinguishable coverage. With mobile Gateways, the partition caused by the bridge failure fell from 90 s to 6.2 s, at the cost of mean coverage falling to 0.922. Flooding delivered more than B.A.T.M.A.N. in open terrain, with about twice the transmissions, but far less in the building (0.430 against 0.786), where its broadcasts have no retries over weak links. Result tables were reproduced byte for byte on two machines. The simulator was not validated against hardware or the batman-adv module, and its radio model is simple: the results describe this implementation under this medium. Those fronts remain as future work.

Keywords: decentralized networks, simulation, B.A.T.M.A.N., mesh networks, coverage, fault tolerance, decentralized mobility, reproducibility.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Contenido

Lista de figuras **11**

Lista de tablas **13**

Lista de símbolos y abreviaturas **15**

Glosario **17**

1. Introducción **19**

2. Planteamiento del problema y objetivos **21**
2.1. Planteamiento del problema 21
2.2. Justificación 22
2.3. Objetivos 23
2.3.1. Objetivo general 23
2.3.2. Objetivos específicos 23
2.4. Alcance y limitaciones 23

3. Estado del arte **25**
3.1. Simuladores de redes de propósito general 25
3.2. Herramientas orientadas a redes descentralizadas 26
3.2.1. Redes ad-hoc móviles y redes malladas 26
3.2.2. Redes entre pares y superposiciones descentralizadas 27
3.2.3. Redes descentralizadas con consenso distribuido 27
3.3. Simulación, emulación y ejecución de código real 27
3.4. El protocolo B.A.T.M.A.N. en la literatura 28
3.5. La credibilidad de los estudios de simulación 29
3.6. Redes de expansión de cobertura 29
3.7. Brecha identificada 30

4. Metodología **33**
4.1. Enfoque 33
4.2. Etapas del trabajo 33
4.3. Requisitos derivados de la brecha 33
4.4. Diseño experimental 35
4.4.1. Despliegue de referencia 35
4.4.2. Experimentos 35

UNIVERSIDAD
SERGIO ARBOLEDA

4.5. Métricas . . . 37
4.6. Análisis estadístico . . . 37
4.7. Verificación del simulador . . . 37
4.8. Amenazas a la validez . . . 39

5. Desarrollo del simulador **41**
5.1. Arquitectura . . . 41
5.2. El protocolo en cada nodo . . . 42
5.3. El modelo del medio . . . 42
5.4. Plano de datos sobre B.A.T.M.A.N. . . . 42
5.5. Movilidad descentralizada de los Gateway . . . 43
5.6. Métricas de descentralización . . . 44
5.7. Generalidad . . . 44
5.8. Escenarios, lotes y modos de ejecución . . . 45
5.8.1. Costo por nodo . . . 45
5.9. Interfaz web . . . 46
5.10. Pruebas . . . 46
5.11. Fidelidad y diferencias con B.A.T.M.A.N. . . . 49

6. Resultados y discusión **51**
6.1. Verificación . . . 51
6.2. Respuesta a los fallos con Gateway fijos . . . 52
6.3. Los mismos fallos con Gateway móviles . . . 55
6.4. Movilidad descentralizada frente a la línea base centralizada . . . 56
6.5. Fallos al azar frente a fallos dirigidos . . . 58
6.6. B.A.T.M.A.N. frente a inundación . . . 59
6.7. Hallazgos sobre el protocolo . . . 62
6.8. Discusión . . . 62

7. Conclusiones y recomendaciones **65**
7.1. Conclusiones . . . 65
7.2. Recomendaciones . . . 66

Bibliografía **69**

A. Reproducción de los resultados **73**
A.1. Preparar el entorno . . . 73
A.2. Correr los experimentos . . . 73
A.3. Generar tablas y figuras . . . 73
A.4. Explorar una corrida . . . 73
A.5. Compilar este documento . . . 74

UNIVERSIDAD
SERGIO ARBOLEDA

Lista de figuras

4.1. Despliegue de referencia en el corte vertical del edificio. El grosor de cada enlace aumenta con su fiabilidad 35
4.2. Fiabilidad de un enlace según la distancia, en el medio por defecto y en el del despliegue 36

5.1. Capas del simulador. La capa de color azul es código del nodo; la de color naranja es lo modelado 41
5.2. Interfaz web durante la caída del Gateway puente (G6). La red está partida y la cobertura sigue completa; los Gateway marcados en rojo tienen alertas activas 47

6.1. Cobertura en el tiempo en cada experimento (media y rango de 10 semillas). Las franjas grises marcan las caídas 53
6.2. Comparación de los experimentos de fallo. Las barras de error indican la desviación estándar 54
6.3. Comparación de los tres modos de movilidad en los cinco escenarios . . 57
6.4. En qué terminó cada paquete enviado, según el escenario y la movilidad 58
6.5. En qué terminó cada paquete enviado, en los experimentos de robustez y de comparación de protocolos 60

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Lista de tablas

3.1. Comparación de las herramientas de simulación y emulación revisadas . 28

4.1. Etapas del desarrollo 34
4.2. Requisitos de un simulador a la medida para redes descentralizadas . . 34
4.3. Experimentos: cada fila es un lote del directorio `lotes/` 36
4.4. Principales métricas de una corrida 38

5.1. Parámetros del medio y del protocolo 43
5.2. Velocidad del simulador según el tamaño de la red (una corrida, semilla 1) 46
5.3. Funciones de la interfaz y vacío de la Sección 3.7 que atienden 48
5.4. Pruebas automatizadas por área 49

6.1. Respuesta de la malla a cada fallo 52
6.2. Calidad de las rutas y del medio en los experimentos de fallo 54
6.3. Respuesta de la malla a cada fallo con Gateway móviles (descentralizada) 55
6.4. Calidad de las rutas y del medio en los experimentos de fallo con Gateway móviles (descentralizada) 55
6.5. Movilidad de los Gateway: *seguir*, *repartir* y *descentralizada* 56
6.6. Plano de datos según la movilidad de los Gateway 57
6.7. Dos caídas al azar contra dos caídas dirigidas al nodo más crítico . . . 59
6.8. B.A.T.M.A.N. contra inundación en dos mapas 59
6.9. Diferencias pareadas por semilla, con intervalo de confianza del 95 % . . 61

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Lista de símbolos y abreviaturas

Símbolos

Símbolo	Significado
a_f	Factor de atenuación por piso de diferencia (`floor_atten`).
d	Distancia entre dos nodos, en metros.
Δf	Diferencia de pisos entre dos nodos.
h	Semiancho de un intervalo de confianza.
LQ	Calidad de enlace: fracción de la ventana de secuencias recibidas de un origen por un vecino.
p_0	Pérdida base del medio (`perdida_base`).
ϕ	Degradación de la fiabilidad con la distancia (`falloff`).
R	Alcance máximo de radio (`rango_comm`), en metros.
$r(d, \Delta f)$	Fiabilidad: probabilidad de entrega de una transmisión.
TQ	Calidad de transmisión acumulada de una ruta.

Abreviaturas

Abreviatura	Término
B.A.T.M.A.N.	*Better Approach To Mobile Ad-hoc Networking*
IC	Intervalo de confianza
MANET	*Mobile Ad-hoc Network*, red ad-hoc móvil
MAC	*Medium Access Control*, control de acceso al medio
OGM	*Originator Message*, mensaje de originador
OLSR	*Optimized Link State Routing*
P2P	*Peer-to-peer*, red entre pares
RPGM	*Reference Point Group Mobility*, movilidad en grupo por punto de referencia
SSE	*Server-Sent Events*

UNIVERSIDAD
SERGIO ARBOLEDA

Abreviatura	**Término**
TTL	*Time To Live*, saltos restantes de un mensaje

UNIVERSIDAD
SERGIO ARBOLEDA

Glosario

BEACON: mensaje periódico con el que un nodo anuncia su identidad y su posición a quienes lo oyen directamente; no se reenvía.

COBERTURA MEDIA: promedio en el tiempo de la fracción de Nodos de usuario vivos que comparten componente de radio con algún Gateway vivo.

ESTIRAMIENTO: saltos que recorrió un paquete entregado divididos por los del camino más corto por radio en el instante del envío.

GATEWAY: nodo de la malla que retransmite tráfico y extiende la cobertura; es el rol que puede moverse para reubicarse.

INUNDACIÓN: protocolo de comparación sin tablas de rutas en el que cada nodo difunde una vez cada paquete nuevo.

LOTE: archivo que define un conjunto de corridas (escenarios por semillas) y del que se obtiene un resumen estadístico.

NODO CRÍTICO: punto de articulación de la malla: si cae, la red queda dividida en más componentes.

NODO DE USUARIO: nodo final que necesita cobertura de la malla.

PLANO DE DATOS: parte del simulador que lleva el tráfico de usuario salto a salto usando la tabla de rutas de cada nodo.

RED DE EXPANSIÓN DE COBERTURA: malla de Gateway desplegada para llevar conectividad a Nodos de usuario en un área, sin coordinador central.

RUTA FANTASMA: ruta vigente en la tabla de un nodo hacia un destino con el que ya no existe ningún camino por radio.

RUTA VIGENTE: ruta presente en la tabla y refrescada por un OGM hace menos que el tiempo límite.

SEMILLA: valor inicial del generador de números aleatorios; con la misma semilla y configuración se repite la misma corrida.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 1

Introducción

Una red descentralizada se define por lo que le falta: no hay un servidor, un controlador ni un punto desde el cual se conozca el estado completo del sistema. Cada nodo decide con la información que le llega, y esa información es parcial y llega tarde. Esta propiedad es lo que hace valiosas a estas redes, porque no dependen de una infraestructura que pueda fallar, y también lo que las hace difíciles de evaluar: el comportamiento que importa no está en ningún nodo, sino en la distancia entre lo que cada uno cree y lo que realmente ocurre.

La evaluación de protocolos de red se apoya mayoritariamente en la simulación, porque desplegar decenas de radios, repetir las condiciones del canal y provocar fallos a voluntad resulta costoso o directamente imposible. Sin embargo, la credibilidad de los estudios de simulación es una preocupación documentada desde hace dos décadas [Pawlikowski et al., 2002, Kurkowski et al., 2005], y se ha mostrado que simuladores distintos pueden dar resultados muy diferentes para un mismo protocolo [Cavin et al., 2002]. A esto se suma un problema de fondo: una herramienta de propósito general evalúa un modelo del protocolo escrito para ella y, además, facilita decidir con información global que ningún nodo real tendría. Un experimento así puede hablar de una red descentralizada sin respetar la condición que la define.

El origen de este trabajo es un simulador de la red B.A.T.M.A.N. orientado a operaciones de búsqueda y rescate [jofsanchezci, 2026]. Al revisarlo con la orientación del tutor, el foco del proyecto se desplazó hacia una pregunta más general: *la necesidad de un simulador hecho a la medida para redes descentralizadas*. La revisión encontró que la herramienta de partida también decidía con información global en tres puntos: los Gateway se movían mirando las posiciones de todos los nodos, los mensajes se entregaban por el camino más corto del grafo real sin consultar las tablas de enrutamiento, y no circulaba tráfico de datos. Corregirlos se volvió parte central del trabajo, y la aplicación concreta pasó a ser una Red de Expansión de Cobertura.

El objetivo de este documento es presentar el diseño, la construcción y la evaluación de ese simulador. La herramienta ejecuta en cada nodo simulado las mismas clases de enrutamiento y de detección de fallos que usa el nodo de red, y sustituye únicamente la capa de transporte por un modelo del medio radio con distancia y pisos. Se evalúa con un despliegue de cobertura en un edificio de tres pisos, sometido a distintos tipos de fallo, con Gateway fijos y móviles, y se compara con líneas base de movilidad centralizada y con un protocolo de inundación.

La metodología combina el desarrollo iterativo guiado por pruebas automatizadas con un diseño experimental de corridas por semillas, en el que cada cifra del documento procede de un lote versionado que puede volver a ejecutarse. El trabajo se limita a

SERGIO ARBOLEDA

un simulador por pasos de tiempo. No incluye un despliegue en hardware, no modela colisiones ni la capa de acceso al medio en detalle, y no mide latencia. Estas limitaciones se tratan de manera explícita a lo largo del documento, porque condicionan qué se puede afirmar a partir de los resultados.

El documento se organiza así. El Capítulo 2 plantea el problema, los objetivos y el alcance. El Capítulo 3 revisa las herramientas existentes y delimita la brecha. El Capítulo 4 describe el método de desarrollo y el diseño experimental. El Capítulo 5 presenta la arquitectura y los mecanismos del simulador. El Capítulo 6 reporta y discute los resultados, y el Capítulo 7 responde a los objetivos y propone líneas futuras. El Anexo A indica cómo reproducir los resultados.

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 2

Planteamiento del problema y objetivos

Este capítulo formula el problema que da origen al trabajo, justifica la construcción de una herramienta propia y fija los objetivos y el alcance.

2.1. Planteamiento del problema

Quien quiere evaluar un protocolo para una red descentralizada dispone de tres tipos de herramienta, y cada uno deja fuera algo que aquí resulta esencial (Capítulo 3). Los simuladores de propósito general, como ns-3 u OMNeT++, evalúan un modelo del protocolo reimplementado dentro de la herramienta, de modo que lo validado no es lo que se ejecutaría en un dispositivo. Los emuladores, como CORE o Mininet-WiFi, ejecutan código real, pero cada nodo consume recursos del anfitrión y el modelo de propagación entre pisos de un edificio no viene incluido. Las herramientas para redes entre pares o de consenso operan en la capa de aplicación y no modelan el medio radio ni el movimiento físico.

A estas diferencias se suma un riesgo más sutil. Un simulador tiene, por construcción, una vista global del sistema: conoce todas las posiciones, todos los enlaces y el camino más corto entre cualquier par de nodos. Si el experimento usa esa información para decidir algo que en la red real decidiría un nodo, ya no está evaluando una red descentralizada. El simulador del que parte este trabajo cometía ese error en tres lugares: la movilidad de los Gateway consultaba las posiciones de todos los nodos, los mensajes se entregaban por búsqueda en el grafo real con garantía de llegada si existía un camino, y no había tráfico de datos que pusiera a prueba las tablas de rutas. Además, la calidad de transmisión (TQ), la métrica central del protocolo, valía siempre 1.0, porque nunca se registraban los mensajes perdidos.

Por otra parte, en una red de este tipo las preguntas relevantes no se responden con una sola cifra de conectividad global: ¿qué cree cada nodo, y cuánto se aleja eso de la realidad?, ¿cuánto tarda la red en notar un fallo y repararse?, ¿qué nodos son críticos y qué pasa si caen precisamente esos?, ¿puede la red reorganizarse sin que nadie la coordine? Responderlas requiere instrumentar la vista de cada nodo y compararla con la verdad física, algo que la herramienta debe ofrecer como parte de su flujo de trabajo.

El problema se formula entonces así:

> *No se dispone de una herramienta que, al mismo tiempo, ejecute el código de enrutamiento de los nodos en lugar de un modelo aparte, no requiera*

UNIVERSIDAD SERGIO ARBOLEDA

virtualizar un sistema por nodo, modele el medio radio de una edificación, garantice que las decisiones de los nodos usen solo información local y mida la divergencia entre lo que cada nodo cree y lo que realmente sucede. Sin ella, la evaluación de redes descentralizadas queda sujeta a modelos que no corresponden a la implementación o a suposiciones de conocimiento global que contradicen el objeto de estudio.

De ahí se derivan las preguntas que orientan el trabajo:

1. ¿Qué requisitos debe cumplir un simulador para evaluar una red descentralizada sin introducir decisiones con información global?

2. ¿Qué revela un simulador así, sobre la cobertura y la resiliencia de una malla B.A.T.M.A.N., que no se aprecia en las métricas de conectividad habituales?

3. ¿Qué ganancias y costos tiene que los Gateway decidan su movimiento solo con lo que oyen, frente a reglas que usan información global?

2.2. Justificación

La justificación es de tres tipos. En lo **científico**, la literatura ha señalado problemas de credibilidad en los estudios de simulación de redes [Kurkowski et al., 2005, Pawlikowski et al., 2002] y una sensibilidad de los resultados a la herramienta empleada [Cavin et al., 2002]. Una herramienta en la que cada corrida se define por un archivo y una semilla, y en la que la frontera entre lo real y lo modelado es explícita, ataca directamente esos problemas.

En lo **práctico**, poder explorar con bajo costo el comportamiento de una malla antes de desplegarla (dónde poner los radios, qué pasa si cae uno, cuántos hacen falta) tiene valor por sí mismo para quien la diseña. Un archivo de escenario y una ejecución de minutos en un computador personal son un costo muy inferior al de un banco de pruebas físico.

En lo **académico**, la interfaz permite ver lo que un nodo cree, su tabla y los mensajes que propagan esa información, lo que convierte a la herramienta en un recurso para enseñar el funcionamiento de las redes descentralizadas. Además, las reglas locales de movilidad conectan este trabajo con el campo de la inteligencia artificial: los Gateway son agentes que, sin coordinación central y con observación parcial, producen un comportamiento colectivo, lo que abre un entorno de pruebas para métodos de aprendizaje de estas reglas (Sección 7.2).

UNIVERSIDAD
SERGIO ARBOLEDA

2.3. Objetivos

2.3.1. Objetivo general

Diseñar, implementar y evaluar un simulador hecho a la medida para redes descentralizadas que ejecute la lógica de enrutamiento B.A.T.M.A.N. de los nodos sobre un modelo del medio radio y permita estudiar, de forma reproducible y solo con decisiones locales, la cobertura y la resiliencia de una Red de Expansión de Cobertura ante fallos de nodos.

2.3.2. Objetivos específicos

1. Caracterizar las herramientas de simulación y emulación aplicables a redes descentralizadas, para delimitar la brecha que justifica una herramienta a la medida.

2. Construir un simulador configurable mediante archivos de escenario que reutilice las clases de enrutamiento y de detección de fallos del nodo de red, con un modelo del medio que contemple la distancia y los pisos de una edificación.

3. Incorporar mecanismos de decisión local (plano de datos salto a salto y movilidad descentralizada de los Gateway) y métricas que cuantifiquen la divergencia entre lo que cree cada nodo y la realidad física.

4. Verificar el comportamiento del simulador mediante pruebas automatizadas, regresión de resultados y reproducibilidad de las corridas.

5. Evaluar una Red de Expansión de Cobertura ante distintos tipos de fallo, comparando la movilidad descentralizada con líneas base centralizadas y B.A.T.M.A.N. con un protocolo de inundación.

2.4. Alcance y limitaciones

Se incluye: un simulador por pasos de tiempo de 0.5 s en Python con interfaz de terminal, ventana propia e interfaz web local; escenarios en un edificio por pisos y en campo abierto; caídas y recuperaciones programables, al azar o dirigidas; tráfico de datos configurable; tres modos de movilidad de Gateway y dos de usuarios; y el análisis de resultados por lotes.

No se incluye: el despliegue físico de la malla; la comparación contra el módulo batman-adv del núcleo de Linux o contra otro simulador; el modelado de colisiones, interferencia o capa MAC detallada; la medición de latencia, porque cada salto se resuelve dentro de un paso; ni la seguridad de la red.

Dos precisiones condicionan la interpretación del trabajo. Primero, el código que se ejecuta es la implementación en Python del nodo de red de este proyecto (paquete `mesh/`), no el módulo batman-adv, y difiere de este en puntos que se documentan en

UNIVERSIDAD SERGIO ARBOLEDA

la Sección 5.11. Segundo, el modelo de radio es una función simple de la distancia y los pisos, de modo que hereda supuestos que se saben optimistas respecto de un canal real [Kotz et al., 2004]. Por ello, los resultados deben leerse como comportamiento de esta implementación bajo este medio, y no como predicción cuantitativa de un despliegue físico.

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 3

Estado del arte

La evaluación de un protocolo de red rara vez se hace desplegando la red completa: el costo, la dificultad de reproducir las condiciones del canal y la imposibilidad de repetir un experimento con exactitud han hecho de la simulación la herramienta dominante en el área. Este capítulo revisa las herramientas de simulación de redes disponibles hoy, distingue las que permiten estudiar redes descentralizadas de las de propósito general y, a partir de esa revisión, delimita el vacío que el presente trabajo pretende cubrir.

3.1. Simuladores de redes de propósito general

La primera generación de herramientas ampliamente adoptadas por la comunidad académica se organiza alrededor del paradigma de *simulación por eventos discretos*: el estado de la red avanza saltando de un evento (transmisión, recepción, expiración de un temporizador) al siguiente, sin recorrer el tiempo de forma continua.

ns-2 y ns-3. El simulador ns-2, desarrollado en el marco del proyecto VINT, fue una herramienta central de la investigación en redes [Breslau et al., 2000]. Su sucesor, ns-3, es un simulador nuevo escrito en C++ [Riley and Henderson, 2010], cuya biblioteca de modelos incluye tecnologías inalámbricas como IEEE 802.11 y LTE [The ns-3 Project, 2026].

OMNeT++ e INET. OMNeT++ es un entorno modular de simulación por eventos discretos escrito en C++, concebido como una infraestructura genérica más que como un simulador de redes cerrado [Varga and Hornig, 2008]. La funcionalidad de red propiamente dicha la aporta el marco de trabajo INET, una biblioteca de modelos que incluye la pila de Internet, protocolos de enlace cableados e inalámbricos, modelos de movilidad y un conjunto de protocolos de enrutamiento para redes ad-hoc.

GloMoSim y QualNet. GloMoSim se propuso específicamente para la simulación paralela de redes inalámbricas de gran escala, organizando la pila de comunicaciones en capas con una API propia por capa, de modo que los modelos de una capa interactúan con los de la capa adyacente únicamente a través de esas interfaces [Zeng et al., 1998]. Su evolución comercial, QualNet, y otras herramientas propietarias como Riverbed Modeler (antes OPNET) y NetSim, ocupan el mismo nicho con soporte comercial y bibliotecas de modelos más extensas.

El rasgo que comparten todas estas herramientas, y que resulta central para este trabajo, es que **el protocolo bajo estudio se reimplementa como un modelo dentro del simulador**. Lo que se valida experimentalmente es, en sentido estricto, ese modelo, no la implementación que después se despliega en un dispositivo real.

UNIVERSIDAD
SERGIO ARBOLEDA

3.2. Herramientas orientadas a redes descentralizadas

El término *red descentralizada* se emplea en la literatura con al menos tres alcances distintos, y a cada uno le corresponde una familia diferente de herramientas. Conviene separarlas antes de compararlas.

3.2.1. Redes ad-hoc móviles y redes malladas

Son redes sin infraestructura fija, en las que cada nodo actúa simultáneamente como terminal y como encaminador. Las herramientas disponibles se reparten entre simuladores y emuladores:

- **ns-3** incluye modelos de los protocolos de enrutamiento ad-hoc clásicos (AODV, OLSR, DSDV y DSR) sobre su modelo de canal inalámbrico [The ns-3 Project, 2026].
- **INET sobre OMNeT++** ofrece un conjunto equivalente de protocolos MANET junto con modelos de movilidad configurables [Varga and Hornig, 2008, INET Framework Project, 2026].
- **Cooja**, el simulador del sistema operativo Contiki para redes de sensores, introduce la noción de *simulación multinivel*: permite simular simultáneamente a nivel de red, de sistema operativo y de instrucción de máquina, ejecutando el firmware real del nodo sobre una emulación del hardware [Österlind et al., 2006]. TOSSIM, del ecosistema TinyOS, simula aplicaciones completas de ese sistema operativo [Levis et al., 2003].
- **EMANE** (*Extendable Mobile Ad-hoc Network Emulator*) es un marco de emulación que usa un modelo de capa física para considerar la propagación de la señal, las antenas y las fuentes de interferencia, con modelos de radio intercambiables [Adjacent Link LLC, 2026].
- **CORE** (*Common Open Research Emulator*) es un emulador en tiempo real que instancia topologías híbridas compuestas por nodos virtuales y hardware real, apoyándose en la virtualización de la pila de red del sistema operativo; escala a topologías de más de un centenar de nodos virtuales sobre un único servidor [Ahrenholz et al., 2008].
- **Mininet-WiFi** extiende Mininet a redes inalámbricas definidas por software: cada nodo es un proceso ubicado en un espacio de nombres de red de Linux, interconectado mediante pares de interfaces virtuales, de manera que cualquier programa que corra sobre Linux corre sin modificaciones dentro de la topología emulada [Fontes et al., 2015].

UNIVERSIDAD
SERGIO ARBOLEDA

3.2.2. Redes entre pares y superposiciones descentralizadas

Cuando la descentralización ocurre en la capa de aplicación y no en la capa física, el objeto de estudio deja de ser el canal inalámbrico y pasa a ser la topología lógica de la superposición (*overlay*). Las herramientas representativas son:

- **PeerSim**, un entorno muy escalable para escenarios dinámicos, en el que los protocolos se implementan específicamente para su API en Java [Montresor and Jelasity, 2009].
- **OverSim**, construido sobre OMNeT++, que combina modelos de superposiciones estructuradas y no estructuradas (Chord, Pastry, Kademlia) con la posibilidad de ejecutar aplicaciones de red reales sobre el mismo código [Baumgart et al., 2009].
- **PlanetSim** [García et al., 2005] y **PeerfactSim.KOM** [Stingl et al., 2011], marcos de simulación de superposiciones y de sistemas entre pares.

3.2.3. Redes descentralizadas con consenso distribuido

Una tercera lectura de *descentralizado* corresponde a los sistemas de registro distribuido. **SimBlock** es un simulador de red de cadena de bloques dirigido por eventos que permite modificar con facilidad el comportamiento de los nodos, de modo que puede estudiarse cómo influye ese comportamiento sobre la red en su conjunto [Aoki et al., 2019]. **BlockSim** es un marco para construir y simular modelos de eventos discretos de sistemas de cadena de bloques [Alharby and van Moorsel, 2020]. Estas herramientas modelan la propagación de bloques y el consenso, pero no la capa física ni la movilidad, por lo que quedan fuera del alcance de este trabajo.

3.3. Simulación, emulación y ejecución de código real

La revisión anterior muestra que las herramientas no se distinguen solamente por el dominio que cubren, sino por un eje más determinante: qué tan cerca está el código evaluado del código que finalmente se despliega. Pueden distinguirse tres niveles:

- **Simulación pura.** El protocolo se reimplementa como modelo dentro del simulador. Escala bien y es perfectamente reproducible, pero introduce una brecha entre el modelo validado y la implementación desplegada.
- **Emulación.** El código real se ejecuta sobre un sistema operativo virtualizado o sobre espacios de nombres de red. La fidelidad es alta, pero cada nodo cuesta recursos del sistema anfitrión y el modelado del medio radio requiere componentes adicionales o hardware específico.

UNIVERSIDAD
SERGIO ARBOLEDA

- **Enfoques híbridos o multinivel.** El código real del protocolo se ejecuta sobre un modelo del medio, como en la simulación multinivel de Cooja [Österlind et al., 2006].

La Tabla 3.1 resume las herramientas revisadas según estos criterios.

Tabla 3.1: Comparación de las herramientas de simulación y emulación revisadas

Herramienta	**Tipo**	**Dominio principal**	**Código del protocolo**
ns-3	Simulador	Redes en general, MANET	Modelo reimplementado
OMNeT++ / INET	Simulador	Redes en general, MANET	Modelo reimplementado
GloMoSim / QualNet	Simulador	Redes inalámbricas a gran escala	Modelo reimplementado
Cooja	Multinivel	Redes de sensores (Contiki)	Firmware real sobre hardware emulado
EMANE	Emulador	MANET, capa física detallada	Código real del sistema
CORE	Emulador	Topologías híbridas en tiempo real	Código real del sistema
Mininet-WiFi	Emulador	Redes inalámbricas definidas por software	Binarios reales de Linux
PeerSim	Simulador	Superposiciones entre pares	Modelo abstracto
OverSim	Simulador	Superposiciones estructuradas	Modelo y aplicaciones reales
SimBlock	Simulador	Redes de cadena de bloques	Modelo de propagación

3.4. El protocolo B.A.T.M.A.N. en la literatura

B.A.T.M.A.N. (*Better Approach To Mobile Ad-hoc Networking*) se especificó como borrador de la IETF, hoy vencido y sin categoría de estándar [Neumann et al., 2008]. A diferencia de un protocolo de estado de enlace como OLSR [Clausen and Jacquet, 2003], ningún nodo construye una visión completa de la topología: cada nodo conoce únicamente cuál de sus vecinos es el mejor siguiente salto hacia cada destino. El borrador de 2008 decide contando mensajes de originador (OGM) recibidos; batman-adv usa una métrica de calidad de transmisión (TQ), derivada de la fracción de OGM propios que un vecino recibe correctamente [Seither et al., 2011].

Su implementación de referencia, *batman-adv*, es un módulo que opera en la capa de enlace [Open Mesh, 2026]. En cuanto a su evaluación, existen estudios experimentales: Murray et al. comparan OLSR, Babel y B.A.T.M.A.N. en una red ad-hoc multisalto,

UNIVERSIDAD SERGIO ARBOLEDA

con Babel como el de mejor desempeño [Murray et al., 2010]; Seither et al. estudian batman-adv en un entorno real de oficina, con AODV como referencia [Seither et al., 2011]; y, más recientemente, Diniz et al. comparan OLSR, batman-adv y Babel para enjambres de drones con dispositivos Raspberry Pi [Diniz et al., 2025]. Estos trabajos dan la referencia más fiel, pero su costo limita la cantidad de nodos, la repetición de las corridas y la posibilidad de provocar fallos a voluntad. La distancia entre simulación y realidad tampoco es menor: Kiess y Mauve recogen que incluso la inundación básica se comportó en prototipos de forma distinta a lo que predecían las simulaciones [Kiess and Mauve, 2007].

3.5. La credibilidad de los estudios de simulación

Elegir la herramienta no es una decisión neutra. Cavin et al. ejecutaron un mismo algoritmo de inundación en tres simuladores y obtuvieron diferencias importantes en los resultados [Cavin et al., 2002]. Pawlikowski et al. alertaron sobre la falta de rigor estadístico en estudios de redes de telecomunicaciones [Pawlikowski et al., 2002], y Kurkowski et al., al revisar las memorias de 2000 a 2005 del simposio MobiHoc, encontraron deficiencias importantes y catalogaron errores comunes en los estudios de simulación de redes móviles [Kurkowski et al., 2005]. Por su parte, Kotz et al. contrastaron supuestos habituales de los modelos de radio con mediciones de un experimento de enrutamiento en exteriores, mostraron su debilidad, entre otras cosas frente a las asimetrías de enlace, y cómo hacen que la simulación difiera de los resultados experimentales [Kotz et al., 2004].

De esta línea se extraen dos exigencias que este trabajo adopta. La primera, que cada resultado sea reproducible a partir de una configuración declarada y que se reporte con su variabilidad. La segunda, que quede explícito qué parte del sistema es un modelo, y qué supuestos del modelo limitan las conclusiones.

3.6. Redes de expansión de cobertura

Las redes ad-hoc multisalto se han estudiado de forma sistemática como respuesta a la falta de comunicaciones cuando la infraestructura fija no existe, está destruida o se encuentra saturada, por ejemplo en zonas de desastre [Reina et al., 2015]. Las propiedades que las hacen adecuadas son el despliegue rápido, la configuración mínima, la conectividad multisalto y la capacidad de auto-reparación ante la pérdida de nodos.

Este trabajo toma de allí su caso de aplicación, llamado aquí *Red de Expansión de Cobertura*: una malla de radios (Gateway) que lleva conectividad a nodos finales (Nodos de usuario) sobre un área, sin un coordinador central. Las preguntas de interés son si los usuarios quedan cubiertos, si la malla sigue conectada y cómo se degrada y se recupera ante un fallo. Tres líneas de trabajo son directamente relevantes:

Autodespliegue con fuerzas virtuales. Howard et al. proponen desplegar una red de sensores móviles en un entorno desconocido mediante campos potenciales, en los

UNIVERSIDAD SERGIO ARBOLEDA

que cada nodo es repelido por los demás y por los obstáculos, con un método distribuido y escalable [Howard et al., 2002]. Poduri y Sukhatme maximizan la cobertura con la restricción de que cada nodo conserve un número mínimo de vecinos [Poduri and Sukhatme, 2004], lo que hace explícito el compromiso entre cubrir más área y mantener la red conectada. Zou y Chakrabarty combinan fuerzas de atracción y repulsión para mejorar la cobertura de un despliegue inicial, en redes organizadas en clústeres, de modo que no se trata de un método plenamente local [Zou and Chakrabarty, 2003]. La idea sigue vigente en redes de drones: Fang et al. la usan en el autodespliegue de grupos de drones para redes de emergencia [Fang et al., 2021], y Liu et al. construyen con ella una red conectada que cubre a usuarios en tierra [Liu et al., 2024]. La movilidad descentralizada de este trabajo (Sección 5.5) pertenece a esta familia.

Modelos de movilidad. Los modelos sintéticos más empleados para estudiar redes ad-hoc, entre ellos el de *random waypoint*, se revisan en [Camp et al., 2002]; para los usuarios que se desplazan juntos existe el modelo de movilidad en grupo por punto de referencia (RPGM) [Hong et al., 1999]. Ambos se incorporaron al simulador para no atar sus resultados a una sola suposición sobre el movimiento de los usuarios.

Fallos al azar y ataques dirigidos. Albert et al. mostraron que las redes libres de escala toleran muy bien la eliminación de nodos al azar y, sin embargo, son vulnerables a la eliminación de sus nodos más conectados [Albert et al., 2000]. Una malla física pequeña no es una red de ese tipo, pero la distinción motiva comparar aquí las dos formas de fallo; para identificar los nodos cuya caída parte la red, se usan los puntos de articulación del grafo, calculables en tiempo lineal con búsqueda en profundidad [Hopcroft and Tarjan, 1973]. Por último, la inundación, el protocolo más simple de difusión, es la comparación natural de un protocolo de rutas, pero se sabe que sus retransmisiones redundantes pueden saturar el medio [Ni et al., 1999].

3.7. Brecha identificada

De la revisión anterior se desprenden cuatro observaciones que, en conjunto, delimitan el aporte de este trabajo:

1. Los simuladores de propósito general y los orientados a MANET (Sección 3.1) evalúan un modelo del protocolo, no su implementación. Las conclusiones obtenidas sobre B.A.T.M.A.N. en esas herramientas son, por tanto, conclusiones sobre el modelo, en general.

2. Los emuladores (Sección 3.2.1) sí ejecutan código real, pero al precio de un costo por nodo considerable en el anfitrión y sin un modelo de propagación tridimensional listo para usar que dé cuenta de la atenuación entre pisos de una edificación, que es precisamente la condición del escenario de interés.

3. Las herramientas de redes descentralizadas de las Secciones 3.2.2 y 3.2.3 operan en la capa de aplicación y no modelan ni el medio radio ni la movilidad física de los nodos.

UNIVERSIDAD
SERGIO ARBOLEDA

4. Hasta donde alcanza esta revisión, medir lo que cada nodo cree frente a lo que ocurre físicamente, o garantizar que las decisiones de los nodos no usen información global, depende de la disciplina de quien instrumenta el experimento: no forma parte del flujo de trabajo habitual de estas herramientas.

Existe entonces un punto intermedio poco explorado: ejecutar la **implementación real del protocolo** —las mismas clases que correrían en un dispositivo desplegado— dentro de un mismo proceso, sustituyendo únicamente la capa de transporte por un modelo del medio radio que sí contemple la geometría del edificio y la atenuación entre pisos. Esta estrategia acerca el código evaluado al que se ejecuta en el nodo, sin virtualizar un sistema por cada uno, y es la que adopta el simulador desarrollado en este trabajo. Conviene precisar que la implementación que se ejecuta aquí es la del nodo en Python de este proyecto, no el módulo batman-adv; sus diferencias se documentan en la Sección 5.11.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 4

Metodología

Este capítulo describe cómo se construyó el simulador y cómo se diseñaron los experimentos que lo evalúan. El propósito es que cualquier cifra del Capítulo 6 pueda rastrearse hasta una configuración declarada.

4.1. Enfoque

El trabajo sigue un enfoque de investigación aplicada, cuantitativo y experimental, en el que el artefacto construido es a la vez un producto del trabajo y el instrumento con el que se obtienen sus resultados. Por eso se le exigen dos condiciones: que su comportamiento esté verificado y que su frontera entre lo implementado y lo modelado sea explícita. El desarrollo fue iterativo, con pruebas automatizadas en cada incremento y una comprobación de regresión sobre los lotes de experimentos cuando un cambio no debía alterar los resultados.

4.2. Etapas del trabajo

La Tabla 4.1 resume las etapas. Conviene destacar dos cambios de encuadre, porque explican varias decisiones técnicas. El proyecto partió de un simulador de búsqueda y rescate [jofsanchezci, 2026]. En septiembre de 2026 pasó a ser una Red de Expansión de Cobertura y se retiró el mecanismo de "hallar supervivientes". Luego, con la indicación del tutor de centrar el problema en *la necesidad de un simulador hecho a la medida para redes descentralizadas*, se revisó la herramienta con ese criterio y se encontraron las tres dependencias de información global que describe la Sección 2.1.

4.3. Requisitos derivados de la brecha

De la brecha de la Sección 3.7 y de las exigencias de la Sección 3.5 se derivan los requisitos de la Tabla 4.2. Son el criterio con el que se juzgó cada decisión de diseño y responden a la primera pregunta del planteamiento.

UNIVERSIDAD
SERGIO ARBOLEDA

Tabla 4.1: Etapas del desarrollo

Periodo (2026)	**Etapa**	**Resultado**
Agosto	Migración del repositorio base	Paquetes `mesh/`, `sim/` y `analysis/`; escenarios en archivos JSON; primeras pruebas
Septiembre	Encuadre de cobertura y cierre de la versión por terminal	Roles Gateway y Nodo de usuario; ejecución sin ventana, semillas y lotes; guía de ejecución
Septiembre y octubre	Interfaz web	Interfaz local con la lente de descentralización, el editor de escenarios y el laboratorio de experimentos
Octubre	Fallos y correcciones del protocolo	Caídas programables, reconvergencia de rutas, corrección del TQ y de la elección de rutas, cobertura media
Octubre	Mecanismos descentralizados	Plano de datos, movilidad descentralizada, métricas de descentralización, campo abierto, movilidad de usuarios, inundación
Octubre y noviembre	Evaluación y documento	Corridas finales, análisis pareado y redacción

Tabla 4.2: Requisitos de un simulador a la medida para redes descentralizadas

Req.	**Requisito**	**Cómo se atiende**
R1	Ejecutar la lógica del protocolo de los nodos, no un modelo aparte	Cada nodo simulado instancia las clases `BatmanRouter` y `FaultManager` de `mesh/`
R2	Hacer explícito lo que se modela	El medio radio es el único reemplazo de los sockets; un panel de la interfaz lo muestra
R3	Modelar el medio de una edificación	Distancia, alcance, pérdida base y atenuación por piso
R4	Decidir solo con información local	Plano de datos por tablas y movilidad basada en lo oído; las reglas centralizadas se conservan solo como línea base
R5	Medir la vista de cada nodo frente a la realidad	Exactitud del conocimiento, rutas fantasma y faltantes, nodos críticos
R6	Permitir repetir cada resultado	Escenario en archivo, semilla, lote versionado y todo el azar en un solo generador
R7	Costo bajo por nodo	Un solo proceso, sin virtualización ni dependencias de red; su alcance real se mide en la Sección 5.8.1

UNIVERSIDAD
SERGIO ARBOLEDA

4.4. Diseño experimental

4.4.1. Despliegue de referencia

El caso principal es un despliegue de 7 Gateway y 6 Nodos de usuario en un edificio de 40 m de ancho y tres pisos de 10 m, representado como un corte vertical (Figura 4.1). Se diseñó para contener los tipos de nodo que importan en una malla: G1, G2, G4 y G5 forman un núcleo redundante en los pisos 1 y 2; G6 es el único puente hacia el piso 3; G7 es el único Gateway al alcance de N6, y G3 el único al alcance de N2.

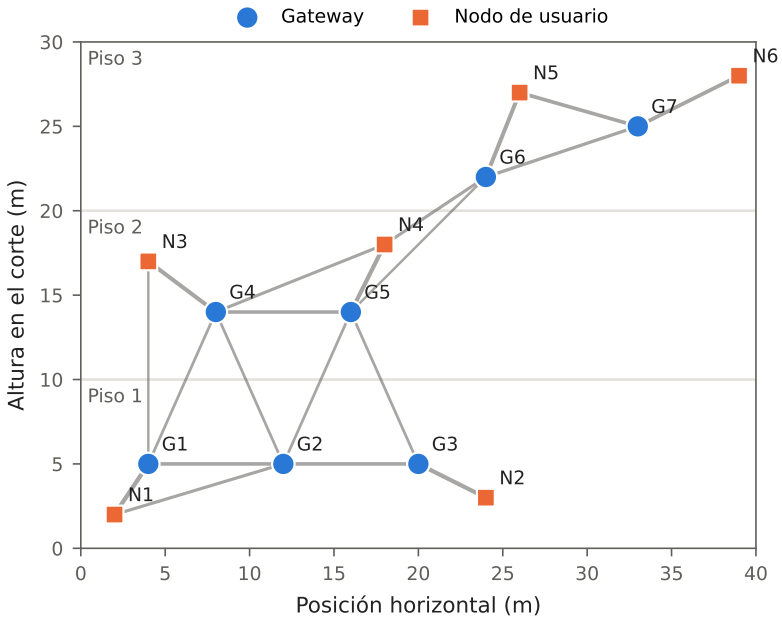


Figura 4.1: Despliegue de referencia en el corte vertical del edificio. El grosor de cada enlace aumenta con su fiabilidad

El medio de este despliegue tiene un alcance de 12 m, una degradación $\phi = 0{,}5$ y una atenuación por piso $a_f = 0{,}8$, que representan radios de baja potencia para interiores. Se eligieron estos valores en lugar de los valores por defecto del simulador (Figura 4.2) por una razón práctica: con el medio por defecto, un despliegue que tenga un puente y un borde en este edificio requiere enlaces tan largos que la entrega cae a 0.32, y las falsas alarmas ocultan los fallos que se quieren estudiar. Esta elección es un supuesto del experimento y no una calibración contra radios reales.

4.4.2. Experimentos

La Tabla 4.3 presenta los experimentos. Cada uno es un lote versionado en el repositorio, que define los escenarios, las semillas y la duración.

UNIVERSIDAD SERGIO ARBOLEDA

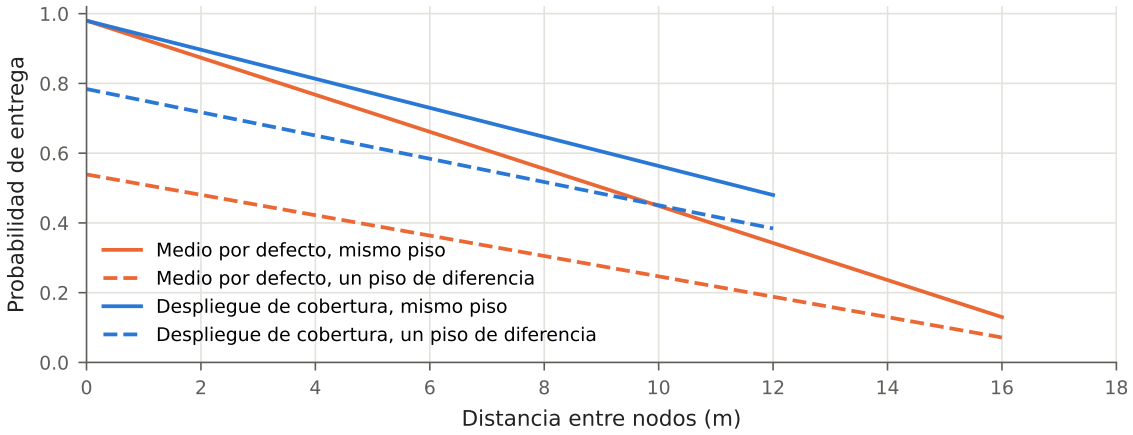


Figura 4.2: Fiabilidad de un enlace según la distancia, en el medio por defecto y en el del despliegue

Tabla 4.3: Experimentos: cada fila es un lote del directorio `lotes/`

Lote	**Qué compara**	**Entradas**	**Semillas**	**Duración (s)**
`capitulo5_fallos`	Control y caídas de un Gateway redundante, un puente, un borde y una cascada, con Gateway fijos	5	10	240
`capitulo5_fallos_movil`	Las mismas caídas con movilidad descentralizada	5	10	240
`movilidad`	*seguir*, *repartir* y *descentralizada* en 5 escenarios	15	10	200
`movilidad_datos`	Los tres modos con 6 flujos de datos, en 3 escenarios	9	10	200
`robustez`	Dos caídas al azar contra dos dirigidas al nodo más crítico, con 6 flujos	2	10	240
`generalidad`	B.A.T.M.A.N. contra inundación en campo abierto (dos movilidades de usuarios) y en el edificio	6	10	300

UNIVERSIDAD
SERGIO ARBOLEDA

En el lote de fallos, el Gateway redundante (G2), el puente (G6) y el de borde (G7) caen a los 60 s y vuelven a los 150 s. En la cascada caen G6 a los 60 s, G7 a los 100 s y G3 a los 140 s, sin vuelta. En total son 42 entradas por 10 semillas, es decir, 420 corridas. Los cinco escenarios heredados del repositorio base (`base`, `colapso_progresivo`, `particion`, `rescatista_perdido` y `denso`) se usan para comparar la movilidad; el nombre `rescatista_perdido` se conserva por trazabilidad y designa simplemente a un Gateway que se aleja del grupo. Los modos *seguir* y *repartir* usan información global a propósito: son la línea base centralizada contra la que se mide lo que se gana o se pierde al decidir localmente.

4.5. Métricas

Cada corrida produce las métricas de la Tabla 4.4. Se distingue entre las que describen la física (qué se puede conectar por radio) y las que describen el protocolo o la creencia de los nodos, porque precisamente esa distancia es el objeto de estudio.

4.6. Análisis estadístico

Cada configuración se corre con 10 semillas (1 a 10) y se reporta como media ± desviación estándar muestral. Las corridas son independientes entre semillas, y en cada corrida la semilla inicializa el único generador de números aleatorios del simulador. Las métricas que no aplican en una corrida no entran en el promedio.

Para comparar configuraciones se emplean diferencias pareadas por semilla: para cada semilla se resta el resultado de las dos configuraciones y se construye un intervalo de confianza del 95 % con la distribución t de Student con 9 grados de libertad. El emparejamiento reduce la variabilidad debida a los flujos sorteados, pero no la elimina, porque las trayectorias aleatorias divergen en cuanto la configuración cambia. No se aplica corrección por comparaciones múltiples, de modo que un intervalo que excluye el cero por poco debe leerse como indicio y no como prueba. Con 10 semillas tampoco se contrasta la normalidad de las diferencias.

4.7. Verificación del simulador

La verificación combina cuatro mecanismos:

- **Pruebas automatizadas** con `unittest` sobre cada capa (Sección 5.10).
- **Regresión de lotes:** un cambio que no debe alterar los resultados se compara, celda por celda, contra la corrida previa de los lotes.
- **Reproducibilidad:** misma semilla y misma configuración deben dar la misma corrida, también entre la terminal y la interfaz web y entre máquinas distintas.

UNIVERSIDAD
SERGIO ARBOLEDA

Tabla 4.4: Principales métricas de una corrida

Métrica	**Definición**
Cobertura media	Promedio en el tiempo de la fracción de Nodos de usuario vivos que comparten componente de radio con algún Gateway vivo
Malla partida (s)	Tiempo con los Gateway vivos en más de un componente. La conectividad se calcula sobre los enlaces de radio entre todos los nodos vivos, de modo que un Nodo de usuario también puede unir dos grupos
Reconvergencia de rutas (s)	Desde que vuelve un Gateway o se reunifica la malla hasta que todo par de Gateway conectados por radio tiene una ruta vigente. “No aplica” si no hubo un episodio cerrado
Alertas	Veces en que un Gateway deja de oír a otro más allá del tiempo límite (30 s)
TQ medio y saltos medios	Promedios sobre las rutas de las tablas de todos los nodos vivos
Tasa de entrega del radio	Transmisiones entregadas sobre intentadas por el medio
Entrega de datos	Paquetes entregados sobre enviados, enrutados por las tablas
Entrega con camino	Lo mismo, solo entre los paquetes con destino vivo y algún camino por radio al enviarse. Aísla el enrutamiento de la física
Estiramiento	Saltos recorridos sobre saltos del camino más corto por radio al enviar
Causas de pérdida	Fracción de los paquetes perdidos sin camino por radio, sin ruta, por ruta vieja, por el salto de radio, o en un bucle (TTL agotado)
Transmisiones por paquete	Intentos unicast o difusiones de datos por cada paquete. No incluye los OGM del plano de control
Exactitud del conocimiento	Fracción de pares ordenados (a, b) de nodos vivos en que a cree lo correcto: tiene ruta vigente si y solo si la radio los conecta
Rutas fantasma y faltantes	Fracción de pares con ruta vigente sin conexión por radio, y con conexión por radio sin ruta vigente
Nodos críticos	Promedio en el tiempo de la cantidad de puntos de articulación del grafo de radio

UNIVERSIDAD
SERGIO ARBOLEDA

- **Contraste analítico:** cuando la métrica se puede calcular a mano a partir del escenario, se compara con la simulación (Sección 6.1).

4.8. Amenazas a la validez

Validez de constructo. La cobertura se define como pertenecer a un componente de radio con algún Gateway, lo que no garantiza un enlace de buena calidad ni que exista una ruta en las tablas. Por eso se reporta junto a la entrega de datos.

Validez interna. Hay parámetros elegidos por los autores: los umbrales de la movilidad descentralizada salieron de una búsqueda en rejilla sobre cuatro de los escenarios en los que luego se compara, y eso favorece a ese modo. El despliegue de referencia se diseñó para exhibir puentes y bordes, de modo que no representa a un despliegue típico.

Validez externa. El modelo de radio es simple y simétrico, sin colisiones ni variación temporal, supuestos que la literatura ha mostrado optimistas [Kotz et al., 2004]. Por ello, las magnitudes absolutas no se extrapolan a un despliegue físico, aunque las comparaciones relativas sí sirven para orientar hipótesis.

Validez de conclusión. Diez semillas dan intervalos amplios en las métricas más variables. En algunas tablas un solo caso domina la dispersión, como se señala al discutirlas.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 5

Desarrollo del simulador

Este capítulo describe el simulador desarrollado: su arquitectura, los mecanismos que lo hacen adecuado para una red descentralizada y la forma de usarlo. El código está escrito en Python y depende solamente de `numpy`, `matplotlib` y `pygame-ce`; la interfaz web utiliza solo la biblioteca estándar y un frontend sin compilación.

5.1. Arquitectura

El sistema se organiza en las capas de la Figura 5.1. La decisión de diseño central es que la capa `mesh/` no sabe que está siendo simulada. La capa `sim/` instancia sus clases en cada nodo y reemplaza lo único que no puede existir en un simulador, los sockets, por un modelo del medio. Todo lo que queda por encima solamente lee ese estado o aplica operaciones sobre él.

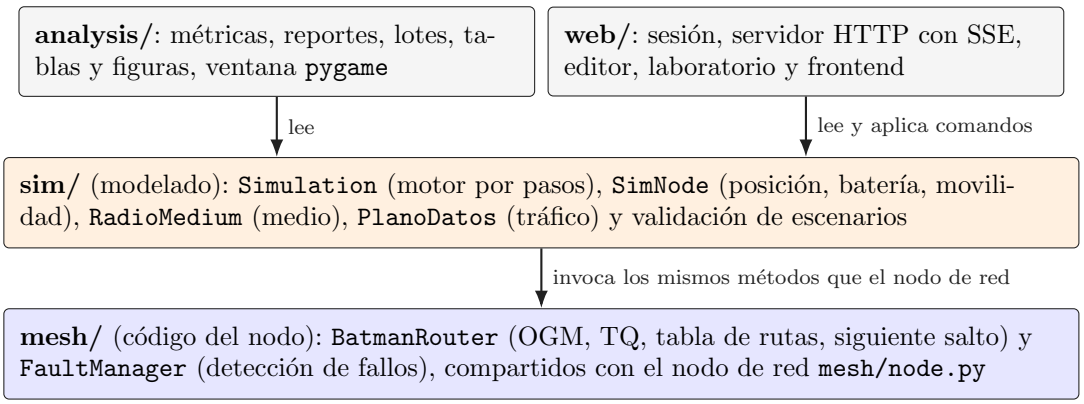


Figura 5.1: Capas del simulador. La capa de color azul es código del nodo; la de color naranja es lo modelado

El motor avanza con un paso fijo de 0.5 s. En cada paso aplica los eventos programados, mueve a los nodos, hace que cada nodo procese lo recibido y emita sus mensajes, envía el tráfico de datos con las tablas ya actualizadas y registra una muestra de las métricas. Todo el azar sale de un único generador, de modo que una semilla y una configuración determinan una corrida completa.

UNIVERSIDAD SERGIO ARBOLEDA

5.2. El protocolo en cada nodo

Cada nodo emite dos tipos de mensaje. El **beacon**, cada 2 s por defecto, anuncia su identidad y su posición a quienes lo oyen directamente. El **OGM**, cada 4 s por defecto, lleva un número de secuencia, un TTL de 6 saltos y un TQ inicial de 1.0, y es retransmitido por quienes lo reciben por primera vez. Todos los nodos vivos reenvían OGM, también los Nodos de usuario.

Calidad de enlace y TQ. Para cada par (origen o, vecino v), `BatmanRouter` mantiene una ventana deslizante de las últimas 16 secuencias: un 1 por cada secuencia recibida y un 0 por cada secuencia saltada. La calidad de enlace y el TQ de la ruta candidata son:

$$LQ_{o,v} = \frac{1}{|W_{o,v}|} \sum_{w \in W_{o,v}} w, \qquad TQ_{o,v} = TQ_{\mathrm{OGM}} \cdot LQ_{o,v}, \tag{5.1}$$

donde TQ_{OGM} es el valor que trae el mensaje, degradado por cada nodo que lo retransmitió.

Elección de ruta. El vecino actual refresca siempre la ruta. Otro vecino la reemplaza solo si su TQ es mayor que el del actual, una vez descontadas las secuencias que el actual ha dejado de entregar. Cualquier copia no vieja mantiene vigente la ruta, porque demuestra que el origen sigue alcanzable.

Detección de fallos. Cada Gateway revisa a sus pares cada 5 s con `FaultManager` y marca como caído a quien lleva más de 30 s sin oír, sin ningún coordinador. Un OGM nuevo o un beacon del nodo apagan la alerta.

5.3. El modelo del medio

`RadioMedium` es el único reemplazo de la red. Cada transmisión se entrega a cada receptor vivo de forma independiente con probabilidad:

$$r(d, \Delta f) = \begin{cases} \max\bigl(0,\ \min\bigl(1,\ [\,1 - \phi\, d/R - p_0\,]\ a_f^{\Delta f}\bigr)\bigr) & \text{si } d \le R \\ 0 & \text{si } d > R \end{cases} \tag{5.2}$$

La Tabla 5.1 muestra los valores empleados. El modelo es deliberadamente simple: es simétrico, no varía en el tiempo y no contempla colisiones. Su aporte frente a un grafo lógico es la atenuación entre pisos y que la pérdida de mensajes haga variar el TQ. La distancia se calcula en el corte vertical del edificio: es un modelo de dos dimensiones con pisos, no una propagación tridimensional completa.

5.4. Plano de datos sobre B.A.T.M.A.N.

En el simulador de partida, un mensaje se entregaba por búsqueda en el grafo real, con garantía de llegada si existía un camino. Las tablas de rutas no decidían nada. Con

UNIVERSIDAD
SERGIO ARBOLEDA

Tabla 5.1: Parámetros del medio y del protocolo

Parámetro	**Por defecto**	**Despliegue**	**Campo abierto**
Alcance de radio R (m)	16	12	30
Degradación con la distancia ϕ	0.85	0.5	0.6
Pérdida base p_0	0.02	0.02	0.02
Atenuación por piso a_f	0.55	0.8	no aplica
Tiempo límite de detección (s)	30	30	30
Periodo de beacons / de OGM (s)	2 / 4	2 / 4	2 / 4
Velocidad de un Gateway (m por paso)	0.32	0 (fijos)	0.6

el plano de datos (`sim/datos.py`), cada paquete avanza salto a salto y en cada nodo el siguiente salto lo decide `BatmanRouter.next_hop()` con la tabla de ese nodo: nadie conoce el camino completo. Cada salto es un envío unicast con hasta 3 reintentos, como en 802.11, sobre la fiabilidad del medio, y el TTL de los datos es 50.

Así, un paquete puede perderse aun habiendo camino, y la causa queda registrada. Las pérdidas se separan en físicas (destino caído, sin ningún camino por radio) y de enrutamiento (sin ruta en la tabla, ruta vieja cuyo siguiente salto ya no está al alcance, salto perdido por la radio y bucle). Esta separación permite no culpar al protocolo de lo que es física. Los flujos se definen en el escenario, con origen, destino y periodo, o se sortean entre pares de usuarios. Cada salto se resuelve dentro del paso de tiempo, porque un salto real dura mucho menos que 0.5 s; por eso el simulador no mide latencia.

5.5. Movilidad descentralizada de los Gateway

Las dos reglas heredadas o agregadas al principio deciden con información global: *seguir* lleva a cada Gateway hacia el Nodo de usuario más cercano mirando las posiciones de todos, lo que termina apilando a los Gateway; y *repartir* es un emparejamiento voraz que calcula el motor. Se conservan solo como línea base.

En la movilidad *descentralizada*, cada Gateway decide únicamente con lo que oyó en los beacons, que llevan la posición del emisor (un supuesto, habitual en estos algoritmos, de que existe un sistema de posicionamiento como GPS o UWB). Olvida a un vecino tras 3 periodos de beacon sin oírlo. La decisión es una suma de fuerzas virtuales, de la familia de los trabajos de autodespliegue revisados en la Sección 3.6, con distancias expresadas como fracción del alcance R:

- **Cobertura.** Atracción hacia los Nodos de usuario que oye y de los que es el Gateway más cercano entre los que conoce (“a su cargo”), si están a más de $0{,}4R$.
- **Repulsión.** Repulsión de los Gateway que oye a menos de $0{,}55R$.

UNIVERSIDAD SERGIO ARBOLEDA

- **Expansión.** Si no tiene usuarios a su cargo, la repulsión llega hasta $0{,}75R$: la malla se estira y explora hasta oír usuarios nuevos.
- **Reconexión.** Si no oye a ningún Gateway, vuelve hacia el último que oyó.

A esas fuerzas se suma una regla de seguridad local: un Gateway no da un paso que rompa un enlace con un Gateway oído (a menos de $0{,}9R$), salvo que quede un rodeo de dos saltos a través de otro Gateway oído. Como las posiciones de los beacons pueden tener varios segundos, cuenta con que cada vecino pudo haberse movido y solo considera seguro un enlace si lo es en el peor caso.

Los umbrales se eligieron con una búsqueda en rejilla sobre cuatro escenarios, con 5 semillas. Esta búsqueda dejó dos aprendizajes. Sin la expansión, Gateway que arrancan juntos y no oyen a nadie se quedan quietos: en `base`, la cobertura era de 0.23. Y una memoria más larga empeoraba las particiones, porque las posiciones viejas engañan más de lo que ayuda recordar al vecino; lo que las redujo fue considerar la incertidumbre de la posición.

5.6. Métricas de descentralización

La función `conocimiento()` compara, en cada muestra y para cada par ordenado de nodos vivos, lo que cree el primero (si tiene una ruta vigente) con la realidad (si la radio los conecta). De ahí salen la exactitud del conocimiento y las rutas fantasma y faltantes de la Tabla 4.4. Una ruta es vigente si un OGM la refrescó hace menos que el tiempo límite; es un criterio necesario, porque las rutas de `BatmanRouter` no expiran solas.

Los nodos críticos se obtienen como puntos de articulación del grafo de radio, con búsqueda en profundidad [Hopcroft and Tarjan, 1973]. Sobre esa base se añadieron los eventos `fail_azar` y `fail_critico`: el nodo que cae se elige al llegar el instante programado, y en el segundo caso es el que deja más nodos separados del grupo de radio más grande. Esto permite una caída dirigida sin fijar de antemano la víctima, lo que tiene sentido sobre todo cuando la red se reorganiza.

5.7. Generalidad

Para que la herramienta no quede atada al edificio de partida se añadieron:

- **Mapa de campo abierto:** un área en planta sin pisos. El escenario `campo_abierto` tiene 120 × 80 m, 6 Gateway que salen del mismo punto, 9 usuarios en 3 grupos y 6 flujos de datos.
- **Movilidad de usuarios:** además de usuarios quietos, *random waypoint* [Camp et al., 2002] y movimiento en grupos inspirado en RPGM [Hong et al., 1999].

UNIVERSIDAD
SERGIO ARBOLEDA

- **Protocolo de comparación:** con `protocolo_datos=inundacion`, cada nodo difunde una vez cada paquete nuevo, sin tablas ni reintentos, porque las difusiones no tienen confirmación.
- **Parámetros en un solo lugar:** cualquier parámetro del medio o del protocolo se puede variar desde la línea de comandos, un lote o la interfaz, con una sola tabla de validación.

5.8. Escenarios, lotes y modos de ejecución

Un escenario es un archivo de texto o JSON que declara el medio, el protocolo, los nodos, los eventos y el tráfico. La validación rechaza con mensajes claros identificadores repetidos, roles inválidos o coordenadas fuera del mapa. Este es un fragmento del escenario de la caída del puente:

```
[medium]
rango_comm=12
falloff=0.5
floor_atten=0.8

[protocol]
static=true

[nodes]
# id  rol  x   y
1    G    4   5
...
6    G    24  22
101  N    2   2

[events]
fail     6  60
recover  6  150
```

La misma simulación se puede ejecutar de cuatro formas, que construyen el mundo con la misma función: una ventana propia con `pygame`; sin ventana con `--headless` o `--inspect`; por lotes con `--batch`, que recorre escenarios por semillas y escribe el resumen estadístico; y en la interfaz con `--web`.

5.8.1. Costo por nodo

La Tabla 5.2 mide la velocidad del simulador con redes aleatorias de distinto tamaño en el mismo edificio, nodos fijos y 6 flujos de datos, en un computador portátil con un

UNIVERSIDAD
SERGIO ARBOLEDA

procesador Intel Core i5-1035G1. El costo crece mucho más rápido que el número de nodos. Al perfilar la corrida de 100 nodos, cerca del 90 % del tiempo se consumió en el procesamiento y reenvío de OGM de cada nodo, y alrededor del 8 % en las métricas: al crecer la red en la misma área, todos se oyen con todos y cada OGM se retransmite muchas veces. Esto marca el rango de uso real de la herramienta: redes de decenas de nodos corren por encima del tiempo real, a partir de unos 100 nodos densos corre por debajo de él, y no es apta para miles de nodos sin optimizarla. Hay que matizar, entonces, el requisito R7: se evita el costo de virtualizar un sistema por nodo, pero un emulador como CORE reporta topologías de más de un centenar de nodos virtuales en tiempo real [Ahrenholz et al., 2008], de modo que la ventaja de esta herramienta frente a la emulación está en el control, la repetibilidad y la instrumentación, y no en la escala.

Tabla 5.2: Velocidad del simulador según el tamaño de la red (una corrida, semilla 1)

Nodos	**Gateway**	**Tiempo por paso (ms)**	**Segundos simulados por segundo real**
14	8	1.8	284.5
25	10	9.2	54.1
50	20	78.6	6.4
100	40	816.7	0.6
200	80	12 006	0.04

5.9. Interfaz web

La interfaz (`python main.py --web`) se sirve desde el mismo proceso y funciona sin conexión a Internet. El navegador solo dibuja lo que recibe: la fiabilidad, las rutas y las métricas se calculan en el servidor con el mismo núcleo, de modo que no hay un segundo modelo que pueda divergir. Se probó que, con la misma semilla, la sesión produce la misma serie de métricas que la ejecución por terminal.

La Figura 5.2 muestra el experimento de la caída del puente a los 99 s simulados: G6 está caído, el indicador avisa de la red partida en dos, y los seis Nodos de usuario siguen cubiertos. La Tabla 5.3 relaciona las funciones de la interfaz con los vacíos identificados en el estado del arte.

5.10. Pruebas

La suite tiene 371 pruebas en `unittest`, distribuidas como indica la Tabla 5.4. Incluyen casos específicos del protocolo (los OGM saltados bajan el TQ, los duplicados no lo alteran y un enlace sin pérdidas se mantiene en 1.0), la equivalencia entre una caída programada y la misma caída aplicada a mano en la interfaz, y que una corrida

UNIVERSIDAD SERGIO ARBOLEDA

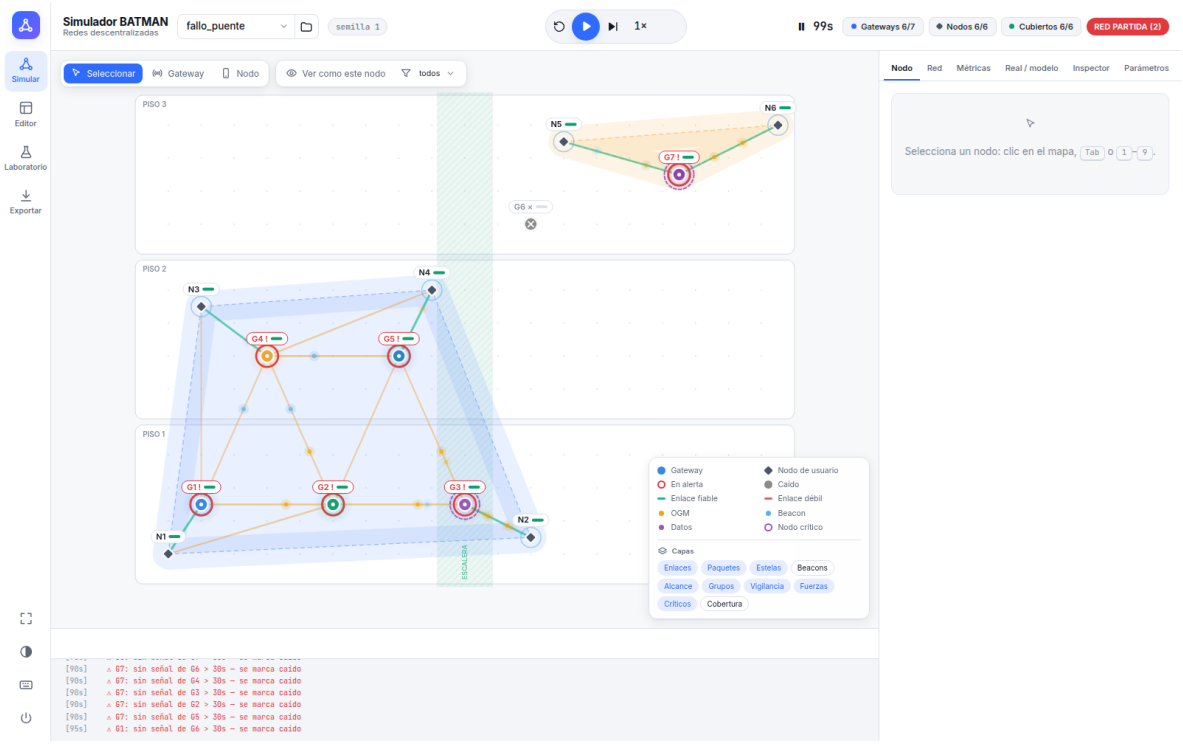


Figura 5.2: Interfaz web durante la caída del Gateway puente (G6). La red está partida y la cobertura sigue completa; los Gateway marcados en rojo tienen alertas activas

UNIVERSIDAD
SERGIO ARBOLEDA

Tabla 5.3: Funciones de la interfaz y vacío de la Sección 3.7 que atienden

Función	**Qué permite**	**Vacío**
Panel de nodo, con rutas y fallos en vivo	Inspeccionar el estado de las clases del protocolo sin instrumentar hardware	1
Panel “Real / modelo”	Ver con contadores qué parte es código del protocolo y qué parte es modelo	1, 2
“Ver como este nodo”	Observar la red solo con lo que sabe un nodo: no hay una vista global para decidir	1, 4
Matriz de conocimiento N×N	Ver cómo se propaga la información y qué rutas sobreviven a una partición	4
OGM de un origen, con el TTL	Seguir la inundación del protocolo salto a salto	1, 3
Enlaces por fiabilidad y mapa de cobertura	Ver el efecto de la distancia y los pisos sobre los enlaces	2, 3
Capas de fuerzas y de nodos críticos	Ver la decisión de cada Gateway y los puntos de articulación	4
Arrastrar nodos y cambiar parámetros en vivo	Experimentar sin desplegar hardware	2
Editor de escenarios y laboratorio de lotes	Pasar de la exploración a resultados con semillas, con los mismos números de la terminal	1, 2
Exportar la sesión	Obtener un escenario y un comando equivalente si solo hubo caídas y recuperaciones	1, 2

UNIVERSIDAD
SERGIO ARBOLEDA

sin tráfico no consuma azar adicional. Se corren con `python -m unittest discover -s tests`.

Tabla 5.4: Pruebas automatizadas por área

Área	**Pruebas**
Protocolo (TQ, elección de rutas) y medio radio	17
Motor y nodos simulados	57
Validación de escenarios	26
Fallos programables y reconvergencia	30
Plano de datos	19
Movilidad descentralizada	13
Métricas de descentralización	12
Generalidad (mapas, movilidad de usuarios, inundación)	14
Reportes, lotes, línea de comandos y análisis	82
Interfaz web	101
Total	**371**

5.11. Fidelidad y diferencias con B.A.T.M.A.N.

Es importante precisar el alcance de la expresión "código real". `BatmanRouter` y `FaultManager` son las clases que usa el nodo de red de este proyecto (`mesh/node.py`, con sockets UDP y TCP), de modo que lo que se evalúa es esa implementación y no un modelo escrito aparte. Pero esa implementación no es el módulo batman-adv, ni se ha ejecutado en este trabajo sobre radios físicas. Las principales diferencias conocidas son:

- **Reenvío de OGM.** Se reenvía la primera copia de cada secuencia, llegue por donde llegue. El protocolo real solo reenvía la que llegó por el mejor vecino. Esto genera bucles reales en las tablas (Sección 6.7).
- **Expiración de rutas.** Las rutas no expiran solas: una ruta por un vecino caído solo se reemplaza cuando su TQ descontado cae por debajo del de otro vecino. El protocolo real purga el originador por tiempo.
- **Reinicio de secuencia.** Un nodo que se recupera conserva su número de secuencia; un arranque en frío queda fuera del modelo, porque los demás ignorarían sus OGM hasta superar la secuencia anterior.

Durante el desarrollo se corrigieron además defectos que afectaban a las conclusiones: el TQ valía siempre 1.0 porque nunca se registraban los OGM perdidos, la primera copia de cada secuencia ganaba la ruta sin mirar el TQ, y una alerta sobre un Gateway a varios saltos nunca se apagaba. Los dos primeros se corrigieron en `mesh/router.py`, y por tanto también valen para el nodo de red.

UNIVERSIDAD SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 6

Resultados y discusión

Este capítulo presenta primero la verificación del simulador y luego los resultados de los experimentos de la Tabla 4.3, con el despliegue de cobertura como caso principal. Salvo indicación contraria, las cifras son media ± desviación estándar de 10 semillas, generadas directamente desde los lotes.

6.1. Verificación

Pruebas. Las 371 pruebas de la Sección 5.10 pasan, con una omitida en Linux porque requiere el motor de JavaScript de macOS.

Reproducibilidad entre máquinas. El 4 de noviembre de 2026 se volvieron a correr todos los lotes en una máquina distinta de aquella en la que se generaron las tablas versionadas el 27 de octubre. Las cinco tablas de esos lotes salieron idénticas byte a byte. Es una comprobación de la capacidad de repetir exactamente los resultados, requisito R6, y no solo de que sean estadísticamente parecidos.

Contraste analítico. Algunos valores se pueden calcular a mano a partir del escenario, y coinciden con la simulación:

- Caída de borde: N6, uno de 6 usuarios, queda sin cobertura durante 90 s de 240 s, de donde $1 - \frac{1}{6} \cdot \frac{90}{240} = 0{,}9375$. La simulación da 0.938.
- Cascada: hay 6 usuarios cubiertos durante 100 s, 4 durante 40 s y 3 durante 100 s, lo que da $\frac{6 \cdot 100 + 4 \cdot 40 + 3 \cdot 100}{6 \cdot 240} = 0{,}736$. La simulación da 0.736, con 3 cubiertos al final.
- Puente: la malla debe estar partida exactamente mientras G6 está caído, 90 s, sin dispersión entre semillas. La simulación da 90.0.

Estos contrastes validan la contabilidad de las métricas físicas, no el comportamiento del protocolo, que se verifica con las pruebas específicas de la Sección 5.10.

Efecto de corregir el TQ. Antes de la corrección, el TQ de toda ruta valía 1.0, por lo que la métrica central del protocolo no informaba nada. Con nodos fijos y semilla 1, el TQ medio pasó a 0.740 en `base`, 0.514 en `denso` y 0.711 en `particion`. Es la clase de defecto que una herramienta sin esta instrumentación puede ocultar: la red parecía perfecta.

UNIVERSIDAD SERGIO ARBOLEDA

6.2. Respuesta a los fallos con Gateway fijos

La Tabla 6.1 resume la respuesta a cada fallo y la Figura 6.1 muestra la evolución de la cobertura.

Tabla 6.1: Respuesta de la malla a cada fallo

Experimento	**Malla partida (s)**	**Reconvergencia de rutas (s)**	**Alertas**	**Cobertura media**	**Nodos cubiertos al final**
Control (sin fallos)	0.0	no aplica	3.1 ± 2.6	1.000	6
Gateway redundante	0.0	7.7 ± 3.7	16.8 ± 2.4	1.000	6
Gateway puente	90.0	19.4 ± 6.7	22.0 ± 1.8	1.000	6
Gateway de borde	0.0	17.1 ± 9.0	11.8 ± 1.7	0.938	6
Cascada	40.0	no aplica	20.1 ± 0.3	0.736	3

Media ± desviación estándar muestral de 10 corridas de 240 s simulados (semillas 1 a 10); sin ±, la desviación es 0. "No aplica": ningún episodio de reconvergencia se cerró.

La cobertura no revela la partición. Es el primer resultado relevante: la caída del puente mantiene la malla partida los 90 s que dura, y sin embargo la cobertura media es de 1.000, porque cada lado conserva algún Gateway. En cambio, la caída de borde no parte la malla y sí reduce la cobertura, a 0.938. Las dos métricas miden daños distintos, y un reporte basado en una sola de ellas daría una imagen falsa del fallo. La Figura 5.2 muestra precisamente ese estado.

Reconvergencia. Tras la vuelta de un Gateway, las rutas tardan 7.7 ± 3.7 s en reconverger si cayó uno redundante, y 19.4 ± 6.7 s si cayó el puente, porque en ese caso deben restablecerse rutas entre dos partes que estuvieron separadas (Figura 6.2). En la cascada no aplica porque ningún nodo vuelve.

Detección y falsas alarmas. La detección distribuida funciona, pero con ruido. La primera alerta llega a los 72.5 ± 9.2 s con la caída del puente y a los 78.0 ± 12.7 s con la del Gateway redundante, es decir, menos de 30 s después de los 60 s de la caída: el tiempo límite se cuenta desde el último OGM oído, y en enlaces con pérdidas ya venía acumulado algo de silencio. El experimento de control muestra el costo de esa sensibilidad: sin ningún fallo hay 3.1 ± 2.6 alertas, y la primera, en 9 de las 10 semillas, a los 126.1 ± 58.6 s. Con enlaces entre 0.41 y 0.65, un tiempo límite de 30 s produce falsas alarmas ocasionales. Esto es un compromiso de diseño que la herramienta permite explorar variando un solo parámetro.

Calidad de las rutas. La Tabla 6.2 indica que el TQ medio es similar entre experimentos, entre 0.32 y 0.36. Es un valor bajo porque el TQ se multiplica salto a salto sobre enlaces de fiabilidad media, con rutas de casi 3 saltos. Que suba ligeramente con los fallos más severos no indica una mejora: al perderse nodos desaparecen las rutas más largas, las de peor TQ, del promedio.

UNIVERSIDAD SERGIO ARBOLEDA

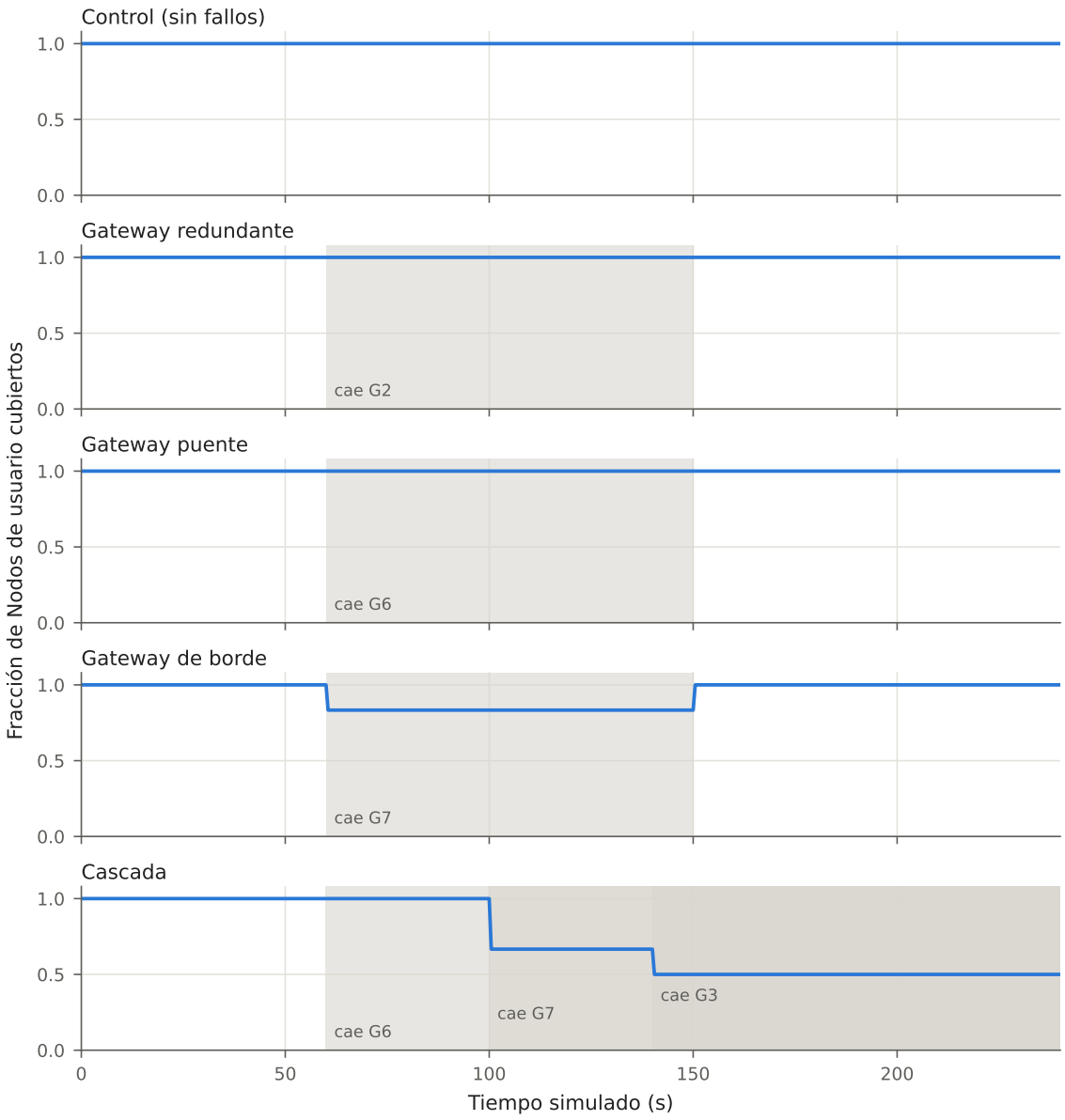


Figura 6.1: Cobertura en el tiempo en cada experimento (media y rango de 10 semillas). Las franjas grises marcan las caídas

UNIVERSIDAD SERGIO ARBOLEDA

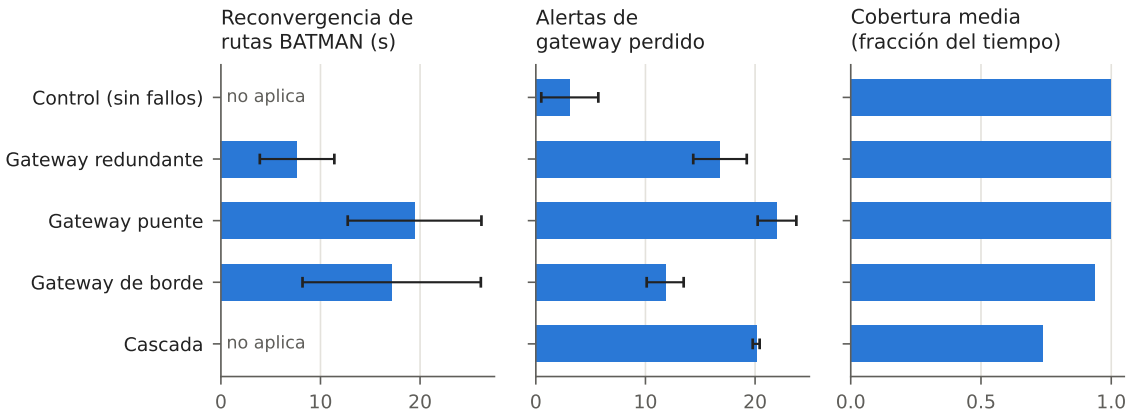


Figura 6.2: Comparación de los experimentos de fallo. Las barras de error indican la desviación estándar

Tabla 6.2: Calidad de las rutas y del medio en los experimentos de fallo

Experimento	**TQ medio de rutas**	**Saltos medios por ruta**	**Tasa de entrega del radio**
Control (sin fallos)	0.331 ± 0.005	2.82 ± 0.03	0.600 ± 0.002
Gateway redundante	0.323 ± 0.004	2.84 ± 0.02	0.606 ± 0.003
Gateway puente	0.339 ± 0.007	2.85 ± 0.03	0.603 ± 0.003
Gateway de borde	0.340 ± 0.007	2.81 ± 0.04	0.599 ± 0.002
Cascada	0.359 ± 0.010	2.87 ± 0.04	0.601 ± 0.003

UNIVERSIDAD SERGIO ARBOLEDA

6.3. Los mismos fallos con Gateway móviles

Las Tablas 6.3 y 6.4 repiten los experimentos con la movilidad descentralizada.

Tabla 6.3: Respuesta de la malla a cada fallo con Gateway móviles (descentralizada)

Experimento	**Malla partida (s)**	**Reconvergencia de rutas (s)**	**Alertas**	**Cobertura media**	**Nodos cubiertos al final**
Control (sin fallos)	0.0	no aplica	6.6 ± 5.6	1.000	6
Gateway redundante	0.0	15.5 ± 9.5	14.5 ± 2.9	1.000 ± 0.001	6
Gateway puente	6.2 ± 6.4	9.7 ± 7.0	14.3 ± 2.7	0.922 ± 0.043	6 ± 0
Gateway de borde	17.0 ± 53.8	30.1 ± 25.5	18.3 ± 7.7	0.959 ± 0.024	6
Cascada	6.2 ± 6.4	4.3 ± 4.0	17.2 ± 2.8	0.826 ± 0.054	4 ± 1

Media ± desviación estándar muestral de 10 corridas de 240 s simulados (semillas 1 a 10); sin ±, la desviación es 0. "No aplica": ningún episodio de reconvergencia se cerró.

Tabla 6.4: Calidad de las rutas y del medio en los experimentos de fallo con Gateway móviles (descentralizada)

Experimento	**TQ medio de rutas**	**Saltos medios por ruta**	**Tasa de entrega del radio**
Control (sin fallos)	0.337 ± 0.010	2.82 ± 0.04	0.605 ± 0.006
Gateway redundante	0.348 ± 0.011	2.77 ± 0.03	0.603 ± 0.004
Gateway puente	0.361 ± 0.011	2.76 ± 0.05	0.597 ± 0.006
Gateway de borde	0.346 ± 0.010	2.81 ± 0.05	0.603 ± 0.006
Cascada	0.364 ± 0.009	2.76 ± 0.05	0.599 ± 0.007

La red se repara sola, con matices. Sin ninguna orden central, los Gateway sobrevivientes se reubican. La partición por la caída del puente baja de 90 s a 6.2 ± 6.4 s, y la cascada deja 4 ± 1 usuarios cubiertos y una cobertura de 0.826, frente a 3 y 0.736 con nodos fijos. Pero moverse también tiene costo. En la caída del puente la cobertura baja de 1.000 a 0.922 ± 0.043, y en la del Gateway redundante la reconvergencia se duplica, de 7.7 s a 15.5 ± 9.5 s, porque la topología sigue cambiando mientras se reparan las rutas. El control también presenta más alertas (6.6 frente a 3.1). No es, por lo tanto, una mejora uniforme, sino un intercambio entre conectividad, cobertura y estabilidad.

Una observación sobre la dispersión: en la caída de borde, el tiempo con la malla partida es de 17.0 ± 53.8 s. Esa media la produce una sola semilla entre diez; no conviene leerla como un comportamiento típico.

UNIVERSIDAD
SERGIO ARBOLEDA

6.4. Movilidad descentralizada frente a la línea base centralizada

La Tabla 6.5 y la Figura 6.3 comparan los tres modos sin tráfico de datos.

Tabla 6.5: Movilidad de los Gateway: *seguir*, *repartir* y *descentralizada*

Escenario	**Movilidad**	**Cobertura media**	**Componentes al final**	**Malla partida (s)**	**Entrega del radio**	**TQ medio**
base	seguir	0.40 ± 0.19	1.1 ± 0.3	16 ± 52	0.86 ± 0.03	0.89 ± 0.03
base	repartir	0.95 ± 0.00	3.0	178 ± 1	0.49 ± 0.01	0.77 ± 0.03
base	descentralizada	0.84 ± 0.11	1.0	5 ± 15	0.27 ± 0.01	0.47 ± 0.04
colapso_progresivo	seguir	0.36 ± 0.07	1.0	5 ± 15	0.80 ± 0.02	0.86 ± 0.03
colapso_progresivo	repartir	0.82 ± 0.00	1.0	102 ± 1	0.47 ± 0.02	0.75 ± 0.03
colapso_progresivo	descentralizada	0.42 ± 0.06	1.0	0	0.27 ± 0.02	0.51 ± 0.05
particion	seguir	0.67	2.0	200	0.96 ± 0.01	0.98 ± 0.01
particion	repartir	0.99 ± 0.00	3.0	200	0.79 ± 0.02	0.88 ± 0.02
particion	descentralizada	0.77 ± 0.13	1.8 ± 0.4	195 ± 12	0.47 ± 0.04	0.55 ± 0.05
rescatista_perdido	seguir	0.91 ± 0.01	2.0	159 ± 0	0.74 ± 0.06	0.79 ± 0.03
rescatista_perdido	repartir	0.93 ± 0.00	2.0	159 ± 0	0.48 ± 0.01	0.70 ± 0.02
rescatista_perdido	descentralizada	0.75 ± 0.08	1.1 ± 0.3	59 ± 17	0.29 ± 0.02	0.56 ± 0.03
denso	seguir	0.51 ± 0.07	1.0	0	0.76 ± 0.03	0.82 ± 0.02
denso	repartir	0.99 ± 0.00	1.7 ± 0.5	85 ± 53	0.35 ± 0.02	0.50 ± 0.05
denso	descentralizada	0.99 ± 0.01	1.0	0	0.30 ± 0.00	0.38 ± 0.01

Media ± desviación estándar de 10 corridas de 200 s simulados (lote `lotes/movilidad.json`). *seguir* y *repartir* son la línea base centralizada; *descentralizada* decide cada Gateway solo con lo que oye.

Los resultados muestran los defectos de cada regla global. Con *seguir*, los Gateway se apilan: la entrega del radio es alta porque los enlaces son cortos, pero la cobertura es baja (0.40 en `base`, 0.51 en `denso`). Con *repartir* la cobertura es casi total, pero la malla pasa partida casi toda la corrida (178 s de 200 s en `base`). La movilidad descentralizada queda en un punto intermedio útil: en `base` y `denso` mantiene una malla casi siempre unida (5 s y 0 s) con cobertura de 0.84 y 0.99. No domina en todo: en `colapso_progresivo` su cobertura (0.42) es muy inferior a la de *repartir* (0.82). Y paga un precio claro: el TQ y la tasa de entrega del radio son los más bajos, porque la malla estirada usa enlaces más largos.

La Tabla 6.6 y la Figura 6.4 añaden tráfico de datos, lo que muestra qué significan esas diferencias para los paquetes.

Según la Tabla 6.9, la movilidad descentralizada entrega más datos que *repartir* en los tres escenarios, y más que *seguir* en `base` y `denso`. En `colapso_progresivo`, la diferencia con *seguir* no es concluyente: el intervalo incluye el cero. Las entregas

UNIVERSIDAD SERGIO ARBOLEDA

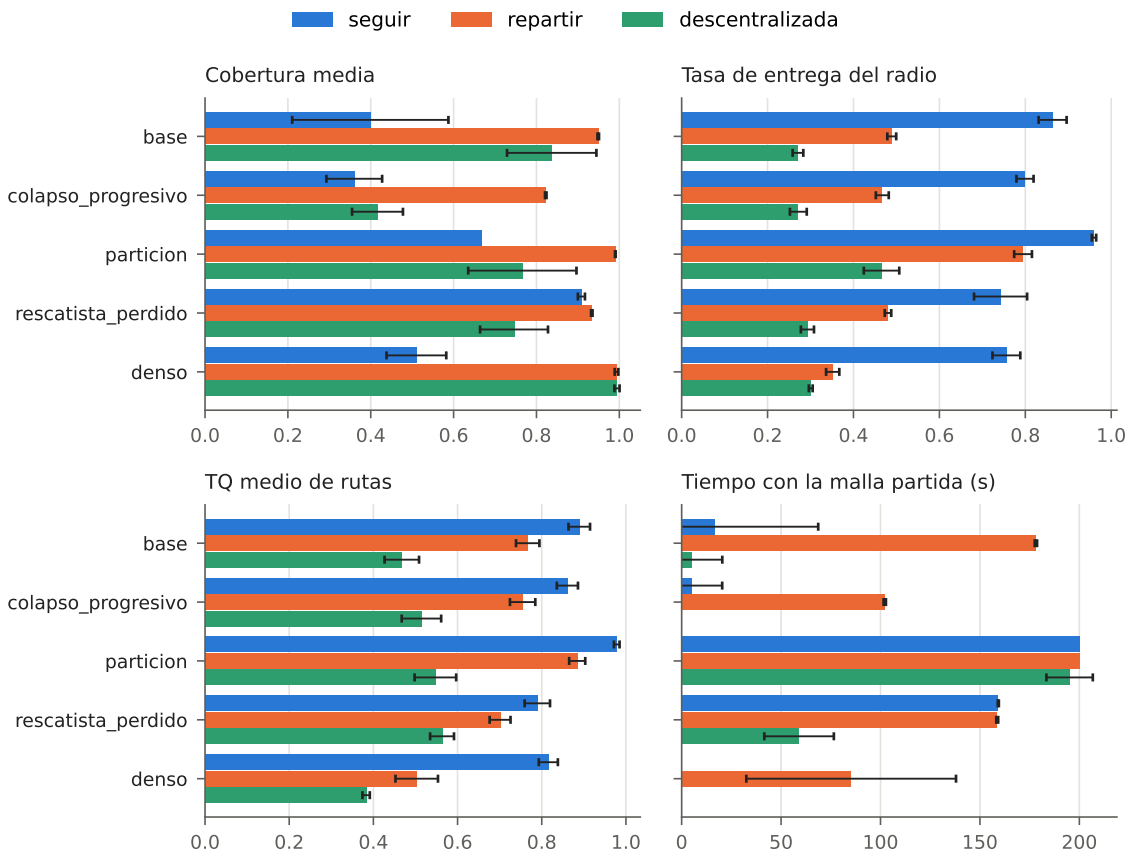


Figura 6.3: Comparación de los tres modos de movilidad en los cinco escenarios

Tabla 6.6: Plano de datos según la movilidad de los Gateway

Escenario	**Movilidad**	**Entrega de datos**	**Entrega con camino**	**Pérdida sin camino**	**Estiramiento**	**Exactitud del conocimiento**
base	seguir	0.21 ± 0.13	0.40 ± 0.15	0.53 ± 0.28	1.04 ± 0.12	0.95 ± 0.02
base	repartir	0.20 ± 0.12	0.35 ± 0.13	0.47 ± 0.24	1.53 ± 0.17	0.90 ± 0.01
base	descentralizada	0.28 ± 0.15	0.30 ± 0.15	0.10 ± 0.05	1.03 ± 0.06	0.55 ± 0.05
colapso_progresivo	seguir	0.22 ± 0.13	0.41 ± 0.15	0.53 ± 0.28	1.01 ± 0.04	0.94 ± 0.03
colapso_progresivo	repartir	0.20 ± 0.12	0.33 ± 0.13	0.47 ± 0.24	1.67 ± 0.23	0.88 ± 0.01
colapso_progresivo	descentralizada	0.23 ± 0.14	0.33 ± 0.16	0.36 ± 0.19	1.00	0.71 ± 0.07
denso	seguir	0.14 ± 0.08	0.30 ± 0.14	0.54 ± 0.14	1.20 ± 0.26	0.89 ± 0.06
denso	repartir	0.18 ± 0.09	0.28 ± 0.08	0.38 ± 0.20	1.41 ± 0.18	0.82 ± 0.07
denso	descentralizada	0.28 ± 0.07	0.28 ± 0.07	0.01 ± 0.01	1.32 ± 0.25	0.86 ± 0.03

Media ± desviación estándar de 10 corridas de 200 s con 6 flujos de datos aleatorios (lote `lotes/movilidad_datos.json`).

UNIVERSIDAD
SERGIO ARBOLEDA

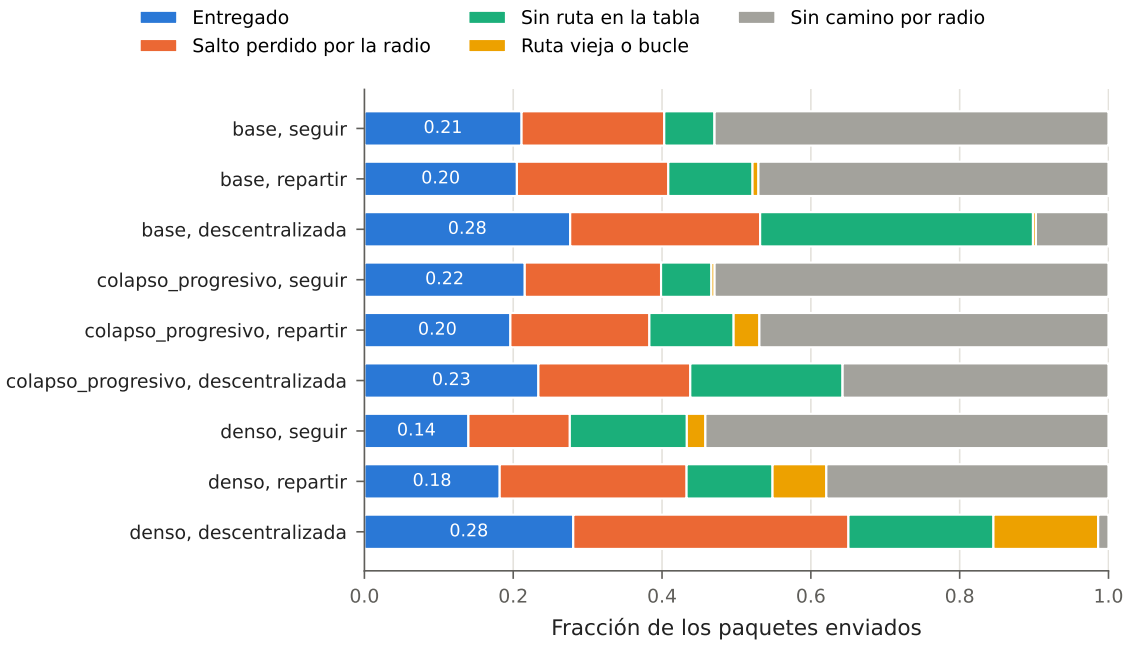


Figura 6.4: En qué terminó cada paquete enviado, según el escenario y la movilidad

absolutas son bajas en todos los casos, entre 0.14 y 0.28. El origen de la ganancia también es instructivo: en la Figura 6.4 se ve que la movilidad descentralizada reduce sobre todo los paquetes sin ningún camino por radio, pero aumenta los perdidos sin ruta y por la radio. Entre pares que sí tienen camino, entrega menos que *seguir* en `base` ($-0{,}104$) y sin diferencia concluyente en los otros dos: gana porque conecta a más pares, no porque enrute mejor.

El plano de control se degrada. Este es el costo más revelador, y solo se aprecia con las métricas de descentralización. En `base`, la exactitud del conocimiento cae de 0.95 con *seguir* a 0.55 con la movilidad descentralizada: en un 44 % de los pares, la radio los conecta y el nodo no tiene una ruta vigente. La malla que la movilidad logra mantener unida físicamente es tan débil que los OGM no alcanzan a mantener las rutas. Una métrica de conectividad física diría que la red está mejor, cuando lo que se tiene es una red conectada que no se conoce a sí misma. Esto sugiere ajustar la movilidad para considerar la calidad de los enlaces y no solo el alcance.

6.5. Fallos al azar frente a fallos dirigidos

La Tabla 6.7 compara dos caídas sorteadas con dos dirigidas al nodo más crítico en el momento de cada caída.

Los fallos dirigidos parten la malla durante 180 s en todas las semillas, frente a 42 ± 57 s al azar, y dejan la entrega de datos en 0.284 frente a 0.502, una diferencia pareada de 0.218 (IC 95 %: 0.149 a 0.287). Recuerda la distinción clásica entre errores

UNIVERSIDAD SERGIO ARBOLEDA

Tabla 6.7: Dos caídas al azar contra dos caídas dirigidas al nodo más crítico

Experimento	**Malla partida (s)**	**Entrega de datos**	**Entrega con camino**	**Cobertura media**	**Rutas fantasma**
Fallos al azar	42 ± 57	0.502 ± 0.114	0.686 ± 0.069	0.925 ± 0.067	0.020 ± 0.013
Fallos dirigidos	180	0.284 ± 0.088	0.696 ± 0.087	0.917	0.052 ± 0.003

Media ± desviación estándar de 10 corridas de 240 s con 6 flujos de datos, caídas a los 60 y 120 s (lote `lotes/robustez.json`).

y ataques [Albert et al., 2000], aunque ese resultado se refiere a redes libres de escala y esta es una malla pequeña y física, de modo que es una analogía y no una confirmación. Lo importante es lo que *no* cambia: la cobertura media (0.925 y 0.917) no los distingue, con un intervalo que incluye el cero, porque cada parte conserva algún Gateway. Tampoco cambia la entrega entre pares con camino (0.686 y 0.696): el enrutamiento sigue funcionando dentro de cada parte y el daño es la falta de camino, que explica el 59.9 % de los paquetes en los fallos dirigidos. Sin plano de datos, este daño no se habría visto con la métrica de cobertura.

6.6. B.A.T.M.A.N. frente a inundación

La Tabla 6.8 y la Figura 6.5 comparan los dos protocolos de datos en dos mapas.

Tabla 6.8: B.A.T.M.A.N. contra inundación en dos mapas

Entorno	**Protocolo**	**Entrega de datos**	**Entrega con camino**	**Transmisiones por paquete**	**Saltos**
Campo abierto, usuarios en grupos	B.A.T.M.A.N.	0.575 ± 0.128	0.803 ± 0.090	2.86 ± 0.91	1.55 ± 0.21
Campo abierto, usuarios en grupos	Inundación	0.607 ± 0.127	0.937 ± 0.042	5.62 ± 1.38	1.56 ± 0.20
Campo abierto, random waypoint	B.A.T.M.A.N.	0.446 ± 0.064	0.544 ± 0.078	3.38 ± 1.08	1.79 ± 0.23
Campo abierto, random waypoint	Inundación	0.618 ± 0.063	0.743 ± 0.071	7.40 ± 1.27	2.24 ± 0.29
Edificio, nodos fijos	B.A.T.M.A.N.	0.786 ± 0.054	0.786 ± 0.054	7.19 ± 1.73	3.90 ± 0.74
Edificio, nodos fijos	Inundación	0.430 ± 0.081	0.430 ± 0.081	6.96 ± 0.34	4.30 ± 0.78

Media ± desviación estándar de 10 corridas de 300 s (lote `lotes/generalidad.json`). B.A.T.M.A.N. reintenta cada salto hasta 3 veces (unicast); la inundación difunde una vez, sin reintentos.

UNIVERSIDAD SERGIO ARBOLEDA

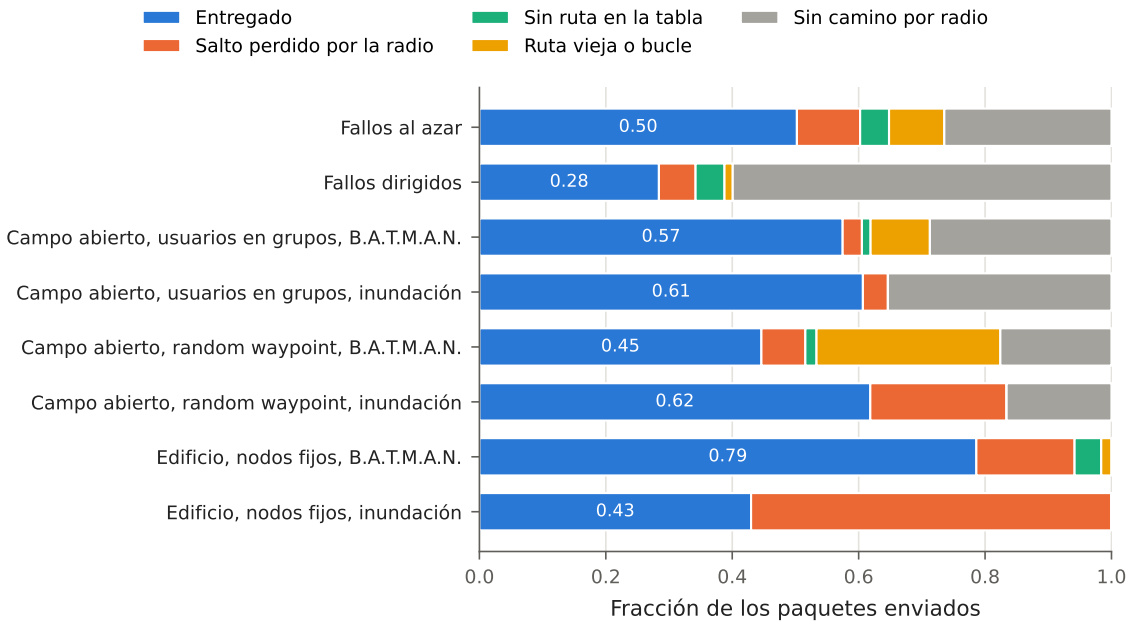


Figura 6.5: En qué terminó cada paquete enviado, en los experimentos de robustez y de comparación de protocolos

La respuesta depende del entorno, y esta es una razón para no atar un simulador a un solo mapa. En campo abierto, la inundación entrega más entre pares con camino que B.A.T.M.A.N. (0.937 frente a 0.803 con usuarios en grupos; 0.743 frente a 0.544 con *random waypoint*), con cerca del doble de transmisiones por paquete. Cuando los usuarios se mueven al azar, la causa principal de pérdida de B.A.T.M.A.N. es la ruta vieja (27.6 % de los paquetes): las tablas no siguen el paso del movimiento. En el edificio ocurre lo contrario: B.A.T.M.A.N. entrega 0.786 frente a 0.430 de la inundación, con un número de transmisiones similar (7.19 y 6.96). Allí, las rutas son largas (casi 4 saltos) sobre enlaces de fiabilidad media, y B.A.T.M.A.N. reintenta cada salto hasta tres veces mientras que la difusión no tiene reintentos, por lo que sus copias se extinguen.

Hay dos precauciones. Primero, las transmisiones por paquete solo cuentan datos, no los OGM que B.A.T.M.A.N. necesita y la inundación no. Segundo, el medio no modela colisiones, de modo que no captura el principal costo conocido de la inundación, la saturación del medio [Ni et al., 1999]. Ambos sesgos actúan en sentidos contrarios, y la comparación es válida para la entrega dentro de este modelo, no como un veredicto sobre los protocolos.

La Tabla 6.9 reúne las diferencias pareadas de los experimentos de este capítulo.

UNIVERSIDAD
SERGIO ARBOLEDA

Tabla 6.9: Diferencias pareadas por semilla, con intervalo de confianza del 95 %

Comparación	**Diferencia media**	**Intervalo del 95 %**	**¿Excluye el 0?**
base: entrega, descentralizada − seguir	+0.065	[+0.020, +0.111]	sí
base: entrega, descentralizada − repartir	+0.072	[+0.032, +0.111]	sí
base: entrega con camino, descentralizada − seguir	−0.104	[−0.195, −0.014]	sí
colapso_progresivo: entrega, descentralizada − seguir	+0.018	[−0.016, +0.052]	no
colapso_progresivo: entrega, descentralizada − repartir	+0.038	[+0.018, +0.058]	sí
colapso_progresivo: entrega con camino, descentralizada − seguir	−0.080	[−0.166, +0.005]	no
denso: entrega, descentralizada − seguir	+0.141	[+0.106, +0.176]	sí
denso: entrega, descentralizada − repartir	+0.099	[+0.060, +0.138]	sí
denso: entrega con camino, descentralizada − seguir	−0.012	[−0.083, +0.060]	no
Robustez: entrega, azar − dirigida	+0.218	[+0.149, +0.287]	sí
Robustez: cobertura, azar − dirigida	+0.008	[−0.040, +0.057]	no
Campo abierto, usuarios en grupos: entrega con camino, inundación − B.A.T.M.A.N.	+0.134	[+0.067, +0.202]	sí
Campo abierto, random waypoint: entrega con camino, inundación − B.A.T.M.A.N.	+0.199	[+0.114, +0.284]	sí
Edificio, nodos fijos: entrega con camino, inundación − B.A.T.M.A.N.	−0.356	[−0.388, −0.325]	sí

Intervalo t de Student con 9 grados de libertad sobre las diferencias entre corridas con la misma semilla. Sin corrección por comparaciones múltiples.

UNIVERSIDAD
SERGIO ARBOLEDA

6.7. Hallazgos sobre el protocolo

Tener el plano de datos sobre las tablas hizo visibles dos comportamientos que antes no se podían observar:

- **Bucles reales.** Entre el 1.3 % y el 2.3 % de los paquetes se pierde en bucles de enrutamiento en los experimentos con B.A.T.M.A.N. del lote de generalidad y de robustez. Su causa probable es que esta implementación reenvía la primera copia de cada OGM, en lugar de solo la del mejor vecino, como lo hace el protocolo.
- **Reparación lenta.** Tras caer un Gateway, la ruta por él permanece en las tablas cerca de 50 s (observado en corridas de examen de `fallo_redundante`), porque solo se reemplaza cuando su TQ descontado cae por debajo del de otra opción. En el experimento de fallos al azar, el 7.4 % de los paquetes se pierde por este motivo. El protocolo real purga el originador por tiempo.

Ninguno de los dos se corrigió, deliberadamente: hacerlo cambiaría el plano de control, y se prefirió reportarlos. Ambos son el tipo de hallazgo que justifica ejecutar el código en lugar de un modelo idealizado: un modelo escrito con la especificación en mente no los habría producido. Y, a la inversa, marcan el límite de los resultados: parte de las pérdidas de este capítulo pertenecen a esta implementación y no a B.A.T.M.A.N. en general.

6.8. Discusión

Volviendo a las preguntas del planteamiento:

Requisitos de la herramienta. La Tabla 4.2 recoge los requisitos; la experiencia indica cuáles fueron decisivos. Separar causas físicas y de enrutamiento, y medir la creencia de los nodos frente a la realidad (R5), explica buena parte de los hallazgos de este capítulo. La regla de decidir solo con información local (R4) no es un purismo: al implantarla aparecieron la degradación del plano de control y los bucles, ocultos mientras los mensajes se entregaban por el grafo real. En cambio, R7 se cumple solo en parte, como muestra la Tabla 5.2.

Qué revela la herramienta. El resultado recurrente es que *una sola métrica de conectividad oculta el daño*: la cobertura no distingue una partición, no separa un ataque de un fallo al azar, y la conectividad física no refleja que los nodos desconocen sus rutas. Para evaluar una red descentralizada hay que medir simultáneamente la física, la entrega y la creencia.

Decidir localmente. La movilidad descentralizada consigue resultados que compiten con las reglas globales sin usar información global, y permite que la malla se repare sola. Sus costos son concretos: enlaces más débiles, un plano de control degradado y más alertas. Esto es coherente con el compromiso entre cobertura y conectividad que formula la literatura de autodespliegue [Poduri and Sukhatme, 2004].

UNIVERSIDAD SERGIO ARBOLEDA

Sobre el alcance de estas conclusiones. Sin validación contra hardware ni contra batman-adv, las magnitudes absolutas no se pueden extrapolar. El medio es optimista en varios aspectos [Kotz et al., 2004] y los umbrales de movilidad se ajustaron sobre parte de los mismos escenarios. Los resultados más sólidos son los de tipo cualitativo y los basados en contrastes con una causa física clara, como el de la cobertura frente a la partición; los más frágiles son las comparaciones de magnitud reducida, sobre todo las diferencias con intervalos cercanos al cero.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 7

Conclusiones y recomendaciones

7.1. Conclusiones

Este trabajo partió de la necesidad de un simulador hecho a la medida para redes descentralizadas y lo construyó, verificó y utilizó para evaluar una Red de Expansión de Cobertura. Las conclusiones se ordenan según los objetivos específicos.

Sobre las herramientas existentes (objetivo 1). La revisión mostró que los simuladores de propósito general evalúan un modelo del protocolo, que los emuladores ejecutan código real pero no traen un modelo listo de atenuación entre pisos, y que las herramientas de redes entre pares y de consenso no modelan el medio radio. Además, se identificó un riesgo propio de cualquier simulador: su vista global facilita decisiones que ningún nodo real podría tomar. El simulador de partida incurría en él en tres lugares, lo que confirma que la condición de descentralización se debe imponer explícitamente.

Sobre la construcción (objetivo 2). Se obtuvo un simulador configurable por archivos, en el que cada nodo ejecuta las clases de enrutamiento y de detección de fallos del nodo de red, con el medio como único reemplazo. Soporta dos tipos de mapa, tres modos de movilidad de Gateway, tres de usuarios contando el reposo, y cuatro formas de ejecución. Su límite es la escala: corre por encima del tiempo real hasta unas decenas de nodos, y por debajo con 100 nodos densos, de modo que su ventaja frente a la emulación es el control y la instrumentación, no el tamaño.

Sobre los mecanismos locales (objetivo 3). El plano de datos y la movilidad descentralizada eliminaron las tres dependencias de información global, las cuales quedan solo como línea base declarada. Las métricas de descentralización aportaron el hallazgo más revelador: en el escenario `base`, la exactitud del conocimiento cae de 0.95 a 0.55 con la movilidad descentralizada, algo que ninguna métrica física mostraba.

Sobre la verificación (objetivo 4). Las 371 pruebas automatizadas, la regresión de lotes, la coincidencia con valores calculados a mano y la reproducción byte a byte de las tablas en dos máquinas respaldan la consistencia interna de la herramienta. Se detectaron y corrigieron defectos de fondo, como un TQ que valía siempre 1.0. Queda sin hacer la validación externa, contra radios físicas o batman-adv, que es la principal deuda del trabajo.

Sobre la evaluación (objetivo 5). Con el despliegue de referencia y 10 semillas se concluye que:

- La cobertura no revela el daño a la conectividad: la caída de un puente parte la malla 90 s con cobertura media de 1.000, y dos fallos dirigidos reducen la entrega de datos de 0.502 a 0.284 sin una diferencia de cobertura concluyente. Evaluar

UNIVERSIDAD SERGIO ARBOLEDA

una red de este tipo exige medir a la vez la física, la entrega y la creencia de los nodos.

- La malla se puede reparar sin coordinador: con Gateway móviles, la partición por la caída del puente baja de 90 s a 6.2 s. El costo es una cobertura de 0.922 en ese mismo fallo, una reconvergencia más lenta tras la caída de un nodo redundante y un plano de control más débil.
- La movilidad descentralizada entrega más datos que *repartir* en los tres escenarios comparados y más que *seguir* en dos de ellos. Lo logra conectando a más pares, no enrutando mejor.
- La comparación entre protocolos depende del entorno: la inundación supera a B.A.T.M.A.N. en campo abierto con cerca del doble de transmisiones, y queda muy por debajo en el edificio (0.430 frente a 0.786). Este resultado se limita a un medio sin colisiones.

En conjunto, el trabajo muestra que la necesidad planteada es real: varios de estos resultados solo aparecieron al ejecutar el código del nodo, imponer decisiones locales y medir la creencia de los nodos, y algunos contradijeron lo que el propio proyecto daba por sentado. Estas conclusiones valen para esta implementación bajo este medio; su extrapolación a un despliegue físico está pendiente de validación.

7.2. Recomendaciones

- **Validación externa.** Es la prioridad. Desplegar `mesh/node.py` en un banco pequeño de radios, repetir los experimentos de fallo y comparar con la simulación; y contrastar una topología contra batman-adv en un emulador o contra un modelo en ns-3.
- **Acercar el protocolo a B.A.T.M.A.N.** Reenviar solo la copia del mejor vecino y purgar originadores por tiempo, y medir cuánto se reducen los bucles y las pérdidas por rutas viejas.
- **Mejorar el medio.** Añadir enlaces asimétricos, variación temporal, colisiones y una capa de acceso al medio, para responder a las críticas conocidas a los modelos simples [Kotz et al., 2004] y para evaluar la inundación con su costo real. Calibrar el medio con mediciones en un edificio.
- **Movilidad consciente de la calidad.** Incluir en las fuerzas la calidad de los enlaces y no solo el alcance, para atacar la degradación del plano de control. Evaluar los umbrales sobre escenarios no usados en su ajuste.
- **Aprendizaje de las reglas locales.** El simulador ofrece un entorno reproducible con agentes de observación parcial, recompensas medibles y un costo por

UNIVERSIDAD
SERGIO ARBOLEDA

corrida bajo. Esto permite plantear, como continuación en inteligencia artificial, el aprendizaje por refuerzo multiagente de la regla de movilidad, comparándola con la de fuerzas virtuales de este trabajo.

- **Escala.** Optimizar el procesamiento de OGM, que domina el tiempo de ejecución, para superar el límite actual de unos 100 nodos densos.
- **Rigor estadístico.** Aumentar el número de semillas en las comparaciones de magnitud reducida y aplicar corrección por comparaciones múltiples.
- **Latencia.** Resolver los saltos con un retardo modelado en lugar de dentro de un paso, lo que permitiría medir la latencia.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Bibliografía

[Adjacent Link LLC, 2026] Adjacent Link LLC (2026). EMANE: Extendable mobile ad-hoc network emulator. https://github.com/adjacentlink/emane. Documentación del proyecto. Consultado en noviembre de 2026. 26

[Ahrenholz et al., 2008] Ahrenholz, J., Danilov, C., Henderson, T. R., and Kim, J. H. (2008). CORE: A real-time network emulator. In *Proceedings of the IEEE Military Communications Conference (MILCOM 2008)*. IEEE. 26, 46

[Albert et al., 2000] Albert, R., Jeong, H., and Barabási, A.-L. (2000). Error and attack tolerance of complex networks. *Nature*, 406(6794):378–382. 30, 59

[Alharby and van Moorsel, 2020] Alharby, M. and van Moorsel, A. (2020). BlockSim: An extensible simulation tool for blockchain systems. *Frontiers in Blockchain*, 3. 27

[Aoki et al., 2019] Aoki, Y., Otsuki, K., Kaneko, T., Banno, R., and Shudo, K. (2019). SimBlock: A blockchain network simulator. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 325–329. IEEE. Preprint disponible en https://arxiv.org/abs/1901.09777. 27

[Baumgart et al., 2009] Baumgart, I., Heep, B., and Krause, S. (2009). OverSim: A scalable and flexible overlay framework for simulation and real network applications. In *Proceedings of the 9th IEEE International Conference on Peer-to-Peer Computing (P2P 2009)*, pages 87–88, Seattle, WA, EE.UU. IEEE. 27

[Breslau et al., 2000] Breslau, L., Estrin, D., Fall, K., Floyd, S., Heidemann, J., Helmy, A., et al. (2000). Advances in network simulation. *Computer*, 33(5):59–67. 25

[Camp et al., 2002] Camp, T., Boleng, J., and Davies, V. (2002). A survey of mobility models for ad hoc network research. *Wireless Communications and Mobile Computing*, 2(5):483–502. 30, 44

[Cavin et al., 2002] Cavin, D., Sasson, Y., and Schiper, A. (2002). On the accuracy of MANET simulators. In *Proceedings of the Second ACM International Workshop on Principles of Mobile Computing*, pages 38–43. 19, 22, 29

[Clausen and Jacquet, 2003] Clausen, T. and Jacquet, P. (2003). Optimized link state routing protocol (OLSR). RFC 3626, Internet Engineering Task Force. 28

[Diniz et al., 2025] Diniz, B. A., Ferrão, I. G., da Silva, L. M., and Branco, K. R. L. J. C. (2025). Comparative performance analysis of OLSR, BATMAN-Adv, and Babel

UNIVERSIDAD SERGIO ARBOLEDA

in UAV mesh networks. In *2025 International Conference on Unmanned Aircraft Systems*, pages 496–503. 29

[Fang et al., 2021] Fang, Y., Cheng, B., Kang, K., and Tan, H. (2021). A self-adaptive deployment model of UAV cluster for emergency communication network. *International Journal of Distributed Sensor Networks*, 17(10). 30

[Fontes et al., 2015] Fontes, R. R., Afzal, S., Brito, S. H. B., Santos, M. A. S., and Rothenberg, C. E. (2015). Mininet-WiFi: Emulating software-defined wireless networks. In *Proceedings of the 11th International Conference on Network and Service Management (CNSM 2015)*, pages 384–389. IEEE. 26

[García et al., 2005] García, P., Pairot, C., Mondéjar, R., Pujol, J., Tejedor, H., and Rallo, R. (2005). PlanetSim: A new overlay network simulation framework. In *Software Engineering and Middleware*, Lecture Notes in Computer Science, pages 123–136. Springer. 27

[Hong et al., 1999] Hong, X., Gerla, M., Pei, G., and Chiang, C.-C. (1999). A group mobility model for ad hoc wireless networks. In *Proceedings of the 2nd ACM International Workshop on Modeling, Analysis and Simulation of Wireless and Mobile Systems*, pages 53–60. 30, 44

[Hopcroft and Tarjan, 1973] Hopcroft, J. and Tarjan, R. (1973). Algorithm 447: Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378. 30, 44

[Howard et al., 2002] Howard, A., Matarić, M. J., and Sukhatme, G. S. (2002). Mobile sensor network deployment using potential fields: A distributed, scalable solution to the area coverage problem. In *Distributed Autonomous Robotic Systems 5*, pages 299–308. Springer, Tokyo. 30

[INET Framework Project, 2026] INET Framework Project (2026). INET framework. `https://inet.omnetpp.org/`. Documentación oficial. Consultado en noviembre de 2026. 26

[jofsanchezci, 2026] jofsanchezci (2026). Simulador_batman: repositorio base del simulador. `https://github.com/jofsanchezci/Simulador_BATMAN`. Rama `develop`. Consultado en agosto de 2026. 19, 33

[Kiess and Mauve, 2007] Kiess, W. and Mauve, M. (2007). A survey on real-world implementations of mobile ad-hoc networks. *Ad Hoc Networks*, 5(3):324–339. 29

[Kotz et al., 2004] Kotz, D., Newport, C., Gray, R. S., Liu, J., Yuan, Y., and Elliott, C. (2004). Experimental evaluation of wireless simulation assumptions. In *Proceedings of the 7th ACM International Symposium on Modeling, Analysis and Simulation of Wireless and Mobile Systems*, pages 78–82. 24, 29, 39, 63, 66

UNIVERSIDAD SERGIO ARBOLEDA

[Kurkowski et al., 2005] Kurkowski, S., Camp, T., and Colagrosso, M. (2005). MANET simulation studies: The incredibles. *ACM SIGMOBILE Mobile Computing and Communications Review*, 9(4):50–61. 19, 22, 29

[Levis et al., 2003] Levis, P., Lee, N., Welsh, M., and Culler, D. (2003). TOSSIM: Accurate and scalable simulation of entire TinyOS applications. In *Proceedings of the 1st International Conference on Embedded Networked Sensor Systems*, pages 126–137. 26

[Liu et al., 2024] Liu, Y., Xie, J., Xing, C., Xie, S., and Luo, X. (2024). Self-organization of UAV networks for maximizing minimum throughput of ground users. *IEEE Transactions on Vehicular Technology*, 73(8):11743–11755. 30

[Montresor and Jelasity, 2009] Montresor, A. and Jelasity, M. (2009). PeerSim: A scalable P2P simulator. In *Proceedings of the 9th IEEE International Conference on Peer-to-Peer Computing (P2P 2009)*, pages 99–100, Seattle, WA, EE.UU. IEEE. 27

[Murray et al., 2010] Murray, D., Dixon, M., and Koziniec, T. (2010). An experimental comparison of routing protocols in multi hop ad hoc networks. In *2010 Australasian Telecommunication Networks and Applications Conference*, pages 159–164. 29

[Neumann et al., 2008] Neumann, A., Aichele, C., Lindner, M., and Wunderlich, S. (2008). Better approach to mobile ad-hoc networking (B.A.T.M.A.N.). IETF Internet-Draft, draft-openmesh-b-a-t-m-a-n-00. Trabajo en progreso. https://datatracker.ietf.org/doc/draft-openmesh-b-a-t-m-a-n/. 28

[Ni et al., 1999] Ni, S.-Y., Tseng, Y.-C., Chen, Y.-S., and Sheu, J.-P. (1999). The broadcast storm problem in a mobile ad hoc network. In *Proceedings of the 5th Annual ACM/IEEE International Conference on Mobile Computing and Networking*, pages 151–162. 30, 60

[Open Mesh, 2026] Open Mesh (2026). batman-adv: B.A.T.M.A.N. advanced, implementación en el núcleo de linux. https://www.open-mesh.org/projects/batman-adv/wiki. Consultado en septiembre de 2026. 28

[Österlind et al., 2006] Österlind, F., Dunkels, A., Eriksson, J., Finne, N., and Voigt, T. (2006). Cross-level sensor network simulation with COOJA. In *Proceedings of the 31st IEEE Conference on Local Computer Networks (LCN 2006)*, pages 641–648. IEEE. 26, 28

[Pawlikowski et al., 2002] Pawlikowski, K., Jeong, H.-D. J., and Lee, J.-S. R. (2002). On credibility of simulation studies of telecommunication networks. *IEEE Communications Magazine*, 40(1):132–139. 19, 22, 29

[Poduri and Sukhatme, 2004] Poduri, S. and Sukhatme, G. S. (2004). Constrained coverage for mobile sensor networks. In *IEEE International Conference on Robotics and Automation, 2004*, pages 165–171. 30, 62

UNIVERSIDAD
SERGIO ARBOLEDA

[Reina et al., 2015] Reina, D. G., Askalani, M., Toral, S. L., Barrero, F., Asimakopoulou, E., and Bessis, N. (2015). A survey on multihop ad hoc networks for disaster response scenarios. *International Journal of Distributed Sensor Networks*, pages 1–16. 29

[Riley and Henderson, 2010] Riley, G. F. and Henderson, T. R. (2010). The ns-3 network simulator. In *Modeling and Tools for Network Simulation*, pages 15–34. Springer, Berlin, Heidelberg. 25

[Seither et al., 2011] Seither, D., König, A., and Hollick, M. (2011). Routing performance of wireless mesh networks: A practical evaluation of BATMAN advanced. In *2011 IEEE 36th Conference on Local Computer Networks*, pages 897–904. 28, 29

[Stingl et al., 2011] Stingl, D., Gross, C., Rückert, J., Nobach, L., Kovacevic, A., and Steinmetz, R. (2011). PeerfactSim.KOM: A simulation framework for peer-to-peer systems. In *2011 International Conference on High Performance Computing and Simulation*, pages 577–584. 27

[The ns-3 Project, 2026] The ns-3 Project (2026). ns-3 model library. https://www.nsnam.org/docs/models/html/routing-overview.html. Documentación oficial. Consultado en noviembre de 2026. 25, 26

[Varga and Hornig, 2008] Varga, A. and Hornig, R. (2008). An overview of the OMNeT++ simulation environment. In *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools '08)*, Marseille, Francia. ICST. 25, 26

[Zeng et al., 1998] Zeng, X., Bagrodia, R., and Gerla, M. (1998). GloMoSim: A library for parallel simulation of large-scale wireless networks. In *Proceedings of the 12th Workshop on Parallel and Distributed Simulation (PADS '98)*, pages 154–161, Banff, Alberta, Canadá. 25

[Zou and Chakrabarty, 2003] Zou, Y. and Chakrabarty, K. (2003). Sensor deployment and target localization based on virtual forces. In *IEEE INFOCOM 2003*, pages 1293–1303. 30

UNIVERSIDAD
SERGIO ARBOLEDA

Anexo A

Reproducción de los resultados

Todas las tablas y figuras del Capítulo 6 se generan a partir de los lotes del repositorio. Se requiere Python 3 con las dependencias de `requirements.txt`.

A.1. Preparar el entorno

```
python3 -m venv venv
venv/bin/pip install -r requirements.txt
venv/bin/python -m unittest discover -s tests
```

A.2. Correr los experimentos

Cada lote deja su salida en `reportes/`: resumen estadístico, una fila por corrida, y los reportes de cada semilla.

```
venv/bin/python main.py --batch lotes/capitulo5_fallos.json
venv/bin/python main.py --batch lotes/capitulo5_fallos_movil.json
venv/bin/python main.py --batch lotes/movilidad.json
venv/bin/python main.py --batch lotes/movilidad_datos.json
venv/bin/python main.py --batch lotes/robustez.json
venv/bin/python main.py --batch lotes/generalidad.json
```

A.3. Generar tablas y figuras

El siguiente comando toma el lote más reciente de cada tipo y reescribe las tablas de `Plantilla/MainMatter/Cap5/tablas/` y las figuras de `Plantilla/Images/Cap5/`. No corre simulaciones.

```
venv/bin/python -m analysis.capitulo5
```

A.4. Explorar una corrida

```
# Interfaz web local, con el escenario de la caída del puente
venv/bin/python main.py --web --seed 1 \
    --config escenarios/fallos/fallo_puente.txt
```

UNIVERSIDAD
SERGIO ARBOLEDA

```
# Una corrida sin ventana, con el resumen en la terminal
venv/bin/python main.py --headless --duracion 240 --seed 1 \
    --config escenarios/fallos/fallo_puente.txt
```

A.5. Compilar este documento

```
cd Plantilla
pdflatex main && bibtex main && pdflatex main && pdflatex main
```

El repositorio incluye una guía más amplia, con casos de uso y sus salidas esperadas, en `docs/guia_ejecucion.md`.

UNIVERSIDAD
SERGIO ARBOLEDA